RFID-BASED IOT ROBOT FOR CUCUMBER PLANT IDENTIFICATION AND PRECISION SPRAYING

P.S.S. Peiris (IT22326386)

T.W.M.J.C. Wijethunga (IT22305978)

C.M.D.D Chandrasekara (IT22925190)

A. H. T. P. P. Thotahewa (IT22892294)

BSc (Hons) degree in Information Technology Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

RFID-BASED IOT ROBOT FOR CUCUMBER PLANT IDENTIFICATION AND PRECISION SPRAYING

P.S.S. Peiris (IT22326386)

T.W.M.J.C. Wijethunga (IT22305978)

C.M.D.D Chandrasekara (IT22925190)

A. H. T. P. P. Thotahewa (IT22892294)

Dissertation submitted in partial fulfilment of the requirements for the Bachelor of Science Special Honors Degree in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

DECLARATION

I declare that this is my own work, and this dissertation does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text. Also, I hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute my dissertation in whole or part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Name	Student ID	Signature
Peiris P.S.S	IT22326386	
Thotabewa A.H.T.P.P	IT22899294	
Wijethunga T.W.M.J.C	IT22305978	
Chandrasekara C.M.D.D	IT22925190	

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

Jeewy ..... 2026/05/12 .....

Signature Of Supervisor — Date

Janaranjana ..... 2026/5/12 .....

Signature Of Co-Supervisor — Date

ACKNOWLEDGEMENT

Several people contributed significantly to the successful completion of this research. First, we would like to express my sincere gratitude to my supervisor, Ms. Wishalya Tissera, and co-supervisor, Ms. Chathushki Chathumali, for their continuous guidance, encouragement, and valuable suggestions throughout this study. Their support and advice were extremely helpful in shaping this research and completing it successfully.

We would also like to thank the Sri Lanka Institute of Information Technology (SLIIT) and the academic staff of the Department of Information Technology for providing the knowledge, support, and academic environment needed to carry out this research effectively.

Our sincere appreciation is also extended to all individuals who supported me during the development and testing of this project, especially those who helped in collecting information, understanding the practical aspects of cucumber cultivation, and giving useful ideas related to smart agriculture and polytunnel management.

In addition, we would like to thank my group members and friends for their cooperation, encouragement, and support throughout the research period.

Finally, we would like to express my deepest gratitude to my family members for their patience, motivation, and unconditional support throughout my academic journey. Their encouragement gave me the strength to complete this study successfully.

ABSTRACT

This research presents an integrated smart cucumber polytunnel management system designed to reduce manual inspection, improve plant-level monitoring, and support precision agricultural decision-making. Cucumber cultivation in protected environments requires continuous observation of plant health, environmental conditions, fruit maturity, and nutrient status. However, traditional practices depend heavily on farmer experience and manual labour, which can delay disease detection, increase chemical wastage, and reduce monitoring accuracy. To address these limitations, the proposed system combines IoT, robotics, RFID identification, computer vision, wireless communication, cloud storage, and mobile application supervision.

The system uses an RFID-based mobile robot to identify individual cucumber plants and capture plant images through a vertical scanning mechanism. This enables plant-specific monitoring because disease detections, spray actions, and scan records can be linked to each plant. An AI-based image analysis module is used to detect healthy leaves, downy mildew, powdery mildew, and water stress. Based on the detection result, the system supports targeted fungicide spraying for fungal diseases while generating alerts for non-fungal conditions such as water stress. Environmental monitoring is implemented using LoRa communication to collect temperature and humidity readings, while anomaly detection compares sensor values to identify unreliable readings. The system also includes YOLOv8-based cucumber detection and ultrasonic-assisted size estimation to support ripeness identification. In addition, NPK monitoring, automated nutrient mixing, and flower detection-based nutrition stage switching are included to improve crop nutrition management.

Firebase is used as the central cloud platform for storing sensor readings, plant records, AI results, spray decisions, ripeness outputs, nutrition data, and alerts. A React Native mobile application allows farmers to monitor system activities and receive important updates remotely. The developed prototype demonstrates the feasibility of integrating plant identification, disease detection, targeted spraying, environmental monitoring, ripeness estimation, and nutrition management into one low-cost smart agriculture solution for cucumber polytunnel cultivation in Sri Lanka.

Keywords:
Smart Agriculture, Cucumber Cultivation, Polytunnel Management, RFID-Based Robot, IoT Monitoring, LoRa Communication, Computer Vision,

Table of Contents

DECLARATION iii
ACKNOWLEDGEMENT iv
ABSTRACT v
LIST OF FIGURES x
List of tables xi
LIST OF ABBREVIATIONS xiii
1. INTRODUCTION 1
1.1. Background and Motivation 1
1.2. Research Gap 2
1.3. Research Problem 4
1.4. Research Aim and Objectives 5
1.4.1. Main Aim 5
1.4.2. Specific Objectives 5
1.5. Scope of the Study 7
1.6. Significance of the Study 7
1.7. Summary 8
2. LITERATURE REVIEW 8
2.1. Introduction 8
2.2. IoT-Based Smart Agriculture Systems 9
2.3. LoRa Communication in Agriculture 9
2.4. RFID-Based Plant Identification 10
2.5. Agricultural Robots and Automated Monitoring 10
2.6. Computer Vision for Disease and Stress Detection 11
2.7. Targeted Spraying Systems 11
2.8. Cucumber Detection and Ripeness Estimation 11
2.9. NPK Monitoring and Nutrition Management 12
2.10. Cloud and Mobile Application Support 12
2.11. Research Gap 13

2.12. Summary 13
3. Methodology 14
3.1. Overall System Architecture 14
3.2. Hardware Architecture 18
3.2.1. Robot Navigation Unit 19
3.2.2. RFID Plant Identification Unit 19
3.2.3. Vertical Image Scanning Unit 20
3.2.4. Image Acquisition Unit 20
3.2.5. Targeted Spraying Unit 21
3.2.6. LoRa-Based Environmental Monitoring Unit 21
3.2.7. Cucumber Ripeness Estimation Hardware 22
3.2.8. NPK Monitoring Unit 23
3.2.9. NPK Mixing Unit 23
3.2.10. Power Supply and Protection 24
3.3. Software Architecture 25
3.3.1. Embedded Control Software 26
3.3.2. AI Model Processing Software 26
3.3.3. LoRa Communication Software 27
3.3.5. React Native Mobile Application 29
3.3.6. Software Data Flow 29
3.4. Dataset Preparation and Data Collection 30
3.4.1. Disease and Water Stress Image Dataset 31
3.4.2. Cucumber Fruit Detection Dataset 33
3.4.3. Flower Detection Dataset 34
3.4.4. Environmental Sensor Data 34
3.4.5. NPK and Nutrition Data 35
3.4.6. Dataset Preprocessing and Augmentation 35
3.4.7. Summary of Prepared Datasets 36
3.5. AI Model Development Methodology 36
3.5.1. Disease and Water Stress Detection Model 37

3.5.2. Disease and Stress Decision Logic 38
3.5.3. Cucumber Fruit Detection Model 38
3.5.4. Cucumber Ripeness Estimation Method 39
3.5.5. Flower Detection Model for Nutrition Season Switching 40
3.5.6. Model Training and Evaluation 41
3.5.7. Integration of AI Models with the Complete System 42
3.6. Robot Operation Workflow 43
3.6.1. Line-Following Navigation 43
3.6.2. RFID-Based Plant Identification 44
3.6.3. Position-Based Plant Scanning 45
3.6.4. Vertical Gantry Scanning 45
3.6.5. Image Capture and Firebase Communication 46
3.6.6. AI Decision Integration 47
3.6.7. Targeted Spraying Operation 47
3.6.8. Robot State-Based Workflow 48
3.6.9. Complete Robot Operation Flow 49
3.7. Disease Detection and Spraying Decision Logic 50
3.7.1. Disease Detection Output Classes 51
3.7.2. Confidence-Based Decision Filtering 51
3.7.3. Disease Severity Estimation 52
3.7.4. Fungicide Selection Logic 52
3.7.5. Spraying Activation Logic 53
3.7.6. Safety and Fail-Safe Logic 54
3.7.7. Firebase-BASED Transfer 54
3.7.8. Complete Disease and Spraying Decision Workflow 55
3.8. LoRa-Based Environmental Monitoring and Sensor Anomaly Detection 56
3.9. Cucumber Detection and Ripeness Estimation Methodology 58
3.10. Nutrition Management Methodology 61
3.11. Firebase and Mobile Application Integration 63
4. Results and Discussion 65

4.1. Introduction 65
4.2. Results 66
4.3. Research Findings 68
4.4. Discussion 71
4.5. Summary 73
4.6. Conclusion 74
4.7. Main Research Contributions 75
4.8. Limitations 77
4.9. Future Enhancements 79
4.10. Final Summary 80
REFERENCES ..... **Error! Bookmark not defined.**

LIST OF FIGURES

Figure 1 Manual Fungicide Spray 4
Figure 2 Problem 5
Figure 3 System Overview 17
Figure 4 Powdery Mildew..... 32
Figure 5 Downey Mildew 32
Figure 6 Healthy Leaf 32
Figure 7 Water Stress 32
Figure 8 Cucumber Fruit 33
Figure 9 Cucumber Flower 34
Figure 10 Robot Operation Flow 50

LIST OF TABLES

Table 1 Research Gap Comparison 2
Table 2 Proposed Group System Contribution 3
Table 3 Main architectural layers 16
Table 4 Problem 18
Table 5 navigation unit 19
Table 6 RFID unit 20
Table 7 The vertical scanning unit 20
Table 8 The spraying unit 21
Table 9 The LoRa monitoring unit 22
Table 10 The ripeness estimation hardware 23
Table 11 The NPK monitoring unit 23
Table 12 The NPK mixing unit. 24
Table 13 Hardware Components in the system 25
Table 14 Firebase structure 28
Table 15 Overall software data flow 30
Table 16 Main Classes of the model classify into 37
Table 17 Severity Levels 38
Table 18 AI Detection Results 38
Table 19 Model Output 39
Table 20 Ripeness Detection 40
Table 21 Flower Detection Stages 41
Table 22 Model Training Metrices 42
Table 23 Robot Line Following Functions 44
Table 24 RFID-BASED PLANT IDENTIFICATION 45
Table 25 VERTICAL GANTRY SCANNING 46
Table 26 AI decisions 47
Table 27 Spray Methods 48
Table 28 Robot Statuses 49
Table 29 DISEASE DETECTION OUTPUT CLASSES 51
Table 30 CONFIDENCE-BASED DECISION FILTERING 52
Table 31 DISEASE SEVERITY ESTIMATION 52
Table 32 FUNGICIDE SELECTION LOGIC 53
Table 33 SAFETY AND FAIL-SAFE LOGIC 54
Table 34 FIREBASE-BASED DECISION TRANSFER 55
Table 35 Sensor Anomaly Detection Logic 57
Table 36 LoRa Monitoring Data Fields 58

Table 37 Cucumber Detection Model Details 59
Table 38 Ripeness Classification Logic 60
Table 39 Cucumber Ripeness Output Data 60
Table 40 Nutrition Management Module Components 62
Table 41 Nutrition Season Switching Logic 63
Table 42 Firebase and Mobile Application Data Integration 64
Table 43 Summary of Main System Results 68
Table 44 Summary of Research Findings 71
Table 45 Summary of Main Contributions 77
Table 46 Summary of System Limitations 78
Table 47 Future Enhancements 80

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
IoT	Internet of Things
RFID	Radio Frequency Identification
LoRa	Long Range Communication
NPK	Nitrogen, Phosphorus, and Potassium
YOLO	You Only Look Once
YOLOv8	You Only Look Once Version 8
CNN	Convolutional Neural Network
Mask R-CNN	Mask Region-Based Convolutional Neural Network
ESP32	Espressif 32-bit Microcontroller
IR	Infrared
RGB	Red, Green, and Blue
OLED	Organic Light Emitting Diode
DC	Direct Current
UI	User Interface
UX	User Experience
API	Application Programming Interface
GPS	Global Positioning System
Wi-Fi	Wireless Fidelity
JSON	JavaScript Object Notation
ML	Machine Learning
mAP	Mean Average Precision
IoU	Intersection over Union
cm	Centimetre

RFID ID	Radio Frequency Identification Identifier
N	Nitrogen
P	Phosphorus
K	Potassium
SLIIT	Sri Lanka Institute of Information Technology

1. INTRODUCTION

1.1. BACKGROUND AND MOTIVATION

Agriculture is a vital part of Sri Lanka's economy, providing food and jobs for many rural communities [1]. Traditional farming depends heavily on the hard work and experience of the farmer. However, modern farming faces new challenges like changing weather, a lack of labor, and the high cost of production. To solve these problems, "Smart Agriculture" uses technology like sensors, robots, and Artificial Intelligence (AI) to help farmers grow crops more efficiently.

Cucumber (Cucumis sativus L.) is one of the most important vegetables grown in Sri Lanka for both local eating and selling. While cucumbers grow well in many areas, they affected by diseases like powdery mildew and downy mildew. To protect these plants, many farmers use polytunnels, which are special plastic houses that control the environment and shield plants from heavy rain and pests.

Even inside a polytunnel, farmers still face many difficulties. Most current smart systems only monitor general things like temperature and humidity. While this is helpful, it does not tell the farmer if a specific plant is sick, if a fruit is ready to pick, or if the soil needs more nutrients. Farmers still have to walk through the tunnel and check every plant by eye. This manual work is very slow, and it is easy to miss the early signs of disease.

The main motivation for this research is to create an integrated system that does the hard work for the farmer. By using a robot that can identify each plant and a smart system that manages nutrients and monitoring, we can reduce the need for manual labor and save money on chemicals.

1.2. RESEARCH GAP

Even though many people are studying smart farming, there is a big "gap" in the solutions available for Sri Lankan farmers. Most existing systems only monitor the environment, like the temperature and humidity of the air. While this information is good, it doesn't tell the farmer if a specific plant is sick or if a cucumber is ready to be picked.

There is also a lack of plant-level identification. Most robots just move through a field without knowing which plant is which. If a robot doesn't know the plant's identity, it cannot keep a history of that plant's health or treatment. Our research fills this gap by using RFID tags to give every plant a "digital ID". Furthermore, many AI systems can detect disease in a photo, but they are not connected to a machine that can actually spray the plant. This research bridges that gap by connecting the AI "brain" directly to a targeted spraying mechanism on the robot.

Area	Traditional/Existing Systems	Proposed Integrated System
Monitoring	General tunnel environment only	Plant-specific health monitoring
Identification	No individual plant tracking	RFID-based digital plant IDs
Treatment	Blanket spraying (wastes chemicals)	Targeted spraying (saves chemicals)
Nutrients	Fixed time-based schedules	Adaptive stage-based mixing

Table 1 Research Gap Comparison

The main problem is that cucumber farming still relies too much on human eyes and hands. In a busy polytunnel, a farmer cannot be everywhere at once. Because cucumber plants grow vertically (upward), they have a lot of leaves at different heights. It is very hard for a farmer to see a disease starting on a leaf high up or hidden behind another branch.

When a farmer finally sees a disease, they usually perform blanket spraying. This means they spray the whole tunnel with chemicals, even the healthy plants. This wastes money, hurts the environment, and can even be bad for the farmer's health. Also, sensors used in smart farms can sometimes break or give wrong data because of the dust and humidity. If a farmer trusts a broken sensor, they might make a mistake that kills the crop.

Area	Existing Systems	Limitation	Proposed Group System Contribution
Environmental monitoring	IoT sensors collect temperature/humidity	Does not directly detect plant condition	LoRa-based monitoring with anomaly detection
Plant identification	Usually no plant-level tracking	Scan/treatment history cannot be linked to individual plants	RFID-based plant identification
Disease detection	AI models classify disease from images	Often not connected to physical spraying	Disease detection connected to targeted spraying
Water stress detection	Often studied separately	Not integrated with disease detection workflow	Disease and water stress considered together
Cucumber detection	Fruit detection models detect cucumber location	Detection may not support harvest decision	YOLOv8 detection + ultrasonic size-based ripeness estimation
Nutrition management	Fixed schedule fertigation	Does not respond to actual crop stage	NPK monitoring + flower-triggered season switching
Communication	Wi-Fi/Bluetooth-based systems	Limited range or infrastructure dependency	LoRa-based sensor transmission
Cloud/mobile system	Data display only	Limited decision support	Firebase + React Native farmer dashboard
Practical local deployment	Many systems are lab-based	Limited Sri Lankan polytunnel suitability	Low-cost integrated prototype for local cucumber growers

Table 2 Proposed Group System Contribution

1.3. RESEARCH PROBLEM

Cucumber cultivation in Sri Lankan polytunnel environments requires continuous monitoring of plant health, environmental conditions, nutrient status, and fruit development. However, in many practical cultivation settings, these tasks are still performed manually. Farmers visually inspect leaves, estimate water stress, check fruit maturity, apply fungicides, and manage nutrients based on experience. This approach can be unreliable because early symptoms may be missed, cucumber fruits may be hidden by leaves, nutrient needs may change before the farmer notices, and environmental sensor faults may go undetected.

The problem becomes more serious because cucumber plants grow vertically and have dense leaf structures. A single fixed observation point cannot cover the entire plant. Diseases such as downy mildew and powdery mildew may appear on leaves at different heights, while water stress may show visible symptoms across the canopy. If detection is delayed, disease can spread to nearby plants and reduce crop performance. Manual blanket spraying may also waste fungicide because healthy plants are treated together with infected plants.



Figure 1 Manual Fungicide Spray

In addition, environmental monitoring alone cannot solve the problem. Temperature and humidity readings help farmers understand polytunnel conditions, but they do not directly identify plant disease, fruit ripeness, nutrient deficiency, or individual plant history. Similarly, AI detection alone is not enough unless it is connected to real-time data storage, farmer alerts, and physical actions such as spraying or nutrient delivery. Therefore, the main challenge is not only detecting problems, but integrating detection, communication, decision-making, actuation, and farmer supervision into one practical system.

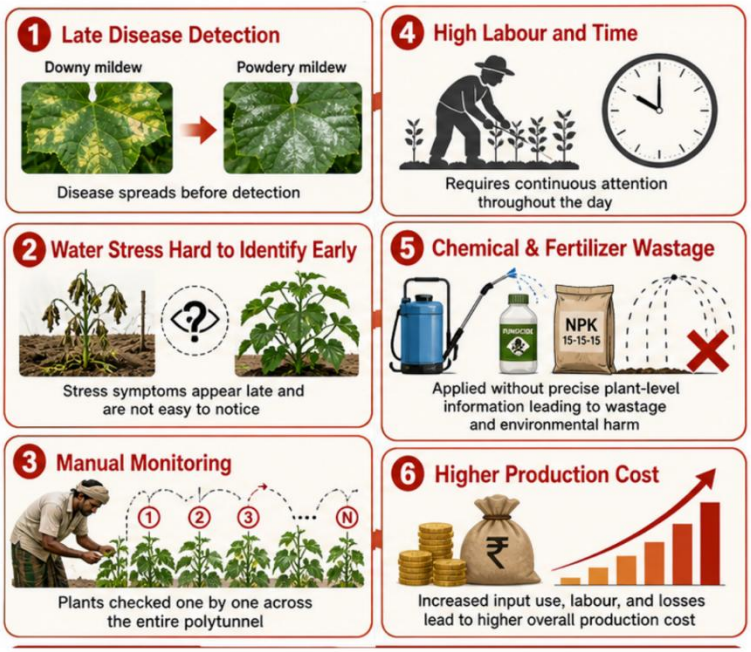


Figure 2 Problem

1.4. RESEARCH AIM AND OBJECTIVES

1.4.1. MAIN AIM

The main aim of this research is to design and develop an integrated smart cucumber polytunnel management system that combines IoT, LoRa communication, RFID-based plant identification, autonomous robotic scanning, computer vision, targeted spraying, sensor anomaly detection, cucumber ripeness estimation, automated nutrient management, and mobile application supervision to support efficient and data-driven cucumber cultivation in Sri Lankan protected agriculture environments.

1.4.2. SPECIFIC OBJECTIVES

1. **To design a mobile robotic platform for cucumber polytunnel navigation.** The robot should move along cucumber rows, stop near plants, and support automated scanning and actuation.
2. **To implement RFID-based plant identification.** Each cucumber plant should be linked to a unique RFID tag so that scan records, disease detections, spray actions, and plant history can be stored accurately.

3. **To develop a vertical scanning mechanism for cucumber plants.**
   Since cucumber plants grow vertically, the system should capture images from multiple plant heights instead of depending on a fixed camera position.

4. **To develop an AI-based cucumber disease and water stress detection module.**
   The system should classify cucumber leaf conditions into healthy, downy mildew, powdery mildew, and water stress categories using image analysis.

5. **To integrate targeted fungicide spraying with AI-based disease decisions.**
   The spraying system should activate only when disease is detected, reducing unnecessary chemical use.

6. **To design a LoRa-based environmental monitoring module.**
   Temperature and humidity readings should be collected from polytunnel sensor nodes and transmitted wirelessly using LoRa communication.

7. **To implement sensor anomaly detection.**
   The system should compare readings from two sensors and identify abnormal differences that may indicate faulty or unreliable sensor data.

8. **To develop a YOLOv8-based cucumber detection and ripeness estimation module.**
   The system should detect cucumber fruits and estimate harvest readiness using image-based dimensions and ultrasonic distance measurements.

9. **To design an automated nutrition management module.**
   The system should monitor NPK values, prepare nutrient solutions using an automated mixer, and deliver nutrients through the drip irrigation line.

10. **To implement flower-detection-based season switching.**
    When cucumber flowers are detected, the system should identify the flowering stage and switch to the relevant nutrient profile.

11. **To integrate Firebase cloud storage and a React Native mobile application.**
    The system should display sensor readings, disease alerts, spray actions, ripeness status, anomaly alerts, nutrient status, and season information in a farmer-friendly interface.

12. **To evaluate the complete system as a low-cost smart agriculture solution.**
    The system should be tested for technical feasibility, usability, and suitability for Sri Lankan cucumber polytunnel cultivation.

1.5. SCOPE OF THE STUDY

This study focuses on the design and development of an integrated smart cucumber polytunnel management prototype. The system is designed for cucumber plants grown inside protected cultivation environments such as polytunnels and greenhouses. The main system scope includes robot navigation, RFID plant identification, vertical image capture, AI-based disease and water stress detection, targeted spraying, LoRa-based environmental monitoring, cucumber detection, ripeness estimation, sensor anomaly detection, NPK monitoring, automated nutrient mixing, flower-triggered nutrition-stage switching, Firebase data storage, and React Native mobile application visualization.

The study is limited to prototype-level implementation and functional validation. It does not aim to develop a fully commercial agricultural robot ready for mass production. The system is tested as proof of concept to demonstrate how different smart agriculture technologies can be combined to support cucumber polytunnel management. Large-scale field trials, long-term agronomic yield comparison, full commercial safety certification, and advanced edge-AI deployment are considered future improvements.

1.6. SIGNIFICANCE OF THE STUDY

The proposed research is significant because it moves beyond isolated smart farming modules and presents an integrated solution for cucumber polytunnel management. Instead of monitoring only environmental conditions, the system observes plant health, plant identity, fruit maturity, nutrient condition, and sensor reliability. This creates a more complete decision-support system for cucumber farmers.

The system can reduce manual labor by allowing the robot to perform plant scanning and targeted treatment support. It can reduce chemical wastage by spraying only when disease is detected. It can improve monitoring accuracy by combining camera-based plant observation with sensor-based environmental readings. It can support harvest decisions by estimating cucumber ripeness based on physical size. It can improve nutrient management by linking flower detection with stage-based nutrient mixing. It can also improve farmer accessibility by displaying important information through a mobile application.

For Sri Lankan protected agriculture, this research contributes to the development of a low-cost, modular, and practical smart farming framework that can be expanded in the future. The system can be adapted for additional crops, more sensors, improved AI models, edge computing, larger polytunnels, and commercial deployment.

1.7. SUMMARY

This chapter introduced the background, motivation, literature foundation, research gaps, research problem, objectives, scope, and significance of the proposed integrated smart cucumber polytunnel management system. The review shows that IoT monitoring, LoRa communication, RFID identification, computer vision, robotic scanning, disease detection, cucumber detection, nutrient monitoring, and mobile application support have each shown value in smart agriculture. However, existing work often treats these areas separately. Therefore, this research proposes a combined system that supports plant-level identification, vertical scanning, AI-based disease and water stress detection, targeted spraying, environmental monitoring, sensor anomaly detection, cucumber ripeness estimation, automated nutrition management, flower-triggered season switching, and mobile-based farmer supervision.

2. LITERATURE REVIEW

2.1. INTRODUCTION

Smart agriculture has become an important solution for improving modern farming practices. It uses technologies such as IoT sensors, wireless communication, robotics, artificial intelligence, cloud databases, and mobile applications to support farmers in monitoring crops more accurately. In protected cultivation environments such as polytunnels, these technologies are useful because crops require continuous monitoring of temperature, humidity, diseases, water stress, nutrient levels, and fruit development.

Cucumber cultivation requires careful observation because cucumber plants grow vertically and have dense leaves. Diseases such as downy mildew and powdery mildew can spread quickly if they are not detected early. Manual inspection is time-consuming and may not always be accurate. Therefore, many researchers have proposed smart farming systems for environmental monitoring, disease detection, automated spraying, fruit detection, and nutrient management. However, most existing systems focus on separate functions rather than one integrated system.

This chapter reviews existing studies related to IoT-based monitoring, LoRa communication, RFID identification, agricultural robots, computer vision-based disease detection, cucumber ripeness estimation, NPK monitoring, and mobile application support.

2.2. IOT-BASED SMART AGRICULTURE SYSTEMS

IoT-based systems are used in agriculture to collect real-time data from sensors. These systems commonly monitor temperature, humidity, soil moisture, light intensity, pH, and soil nutrients. In greenhouse and polytunnel farming, IoT monitoring helps farmers understand environmental conditions without checking the field manually.

Existing IoT greenhouse systems usually connect sensors to microcontrollers and send the collected data to a cloud platform or mobile application. This allows farmers to view environmental readings remotely. These systems reduce manual work and improve decision-making related to irrigation, ventilation, and crop protection.

However, many IoT systems only monitor general environmental conditions. They do not directly identify whether a specific plant is infected, whether a cucumber is ready to harvest, or whether a nutrient profile should be changed. Therefore, IoT monitoring alone is not enough for complete cucumber polytunnel management.

2.3. LORA COMMUNICATION IN AGRICULTURE

LoRa communication is useful in agriculture because it supports long-range, low-power wireless data transmission. It is suitable for transmitting small sensor readings such as temperature and humidity. Compared with Wi-Fi or Bluetooth, LoRa can work over longer distances and is useful in farming areas where internet coverage is limited.

In smart agriculture, LoRa is commonly used to send sensor data from field nodes to a receiver or gateway. The gateway can then upload the data to a cloud database for monitoring. This makes LoRa suitable for polytunnel environmental monitoring.

However, LoRa is mainly suitable for small data packets. It is not suitable for sending large image files or video streams. Therefore, LoRa-based systems are usually limited to sensor monitoring. To create a complete smart farming system, LoRa monitoring should be combined with image processing, disease detection, mobile alerts, and decision-making.

2.4. RFID-BASED PLANT IDENTIFICATION

RFID technology is used to identify objects using RFID tags and readers. In agriculture, RFID can be used to identify plants, trays, animals, or equipment. Plant-level identification is useful because each plant can have a unique digital identity.

In cucumber cultivation, RFID can help link scan results, disease records, spray actions, and plant history to a specific plant. This is better than monitoring only the general tunnel condition. Without plant-level identification, the system cannot know which exact plant was scanned or treated.

Existing smart agriculture systems often collect environmental data but do not identify individual plants. This limits traceability. RFID can solve this issue, but RFID alone cannot detect disease or control spraying. Therefore, RFID should be integrated with robotics, computer vision, Firebase storage, and mobile monitoring.

2.5. AGRICULTURAL ROBOTS AND AUTOMATED MONITORING

Agricultural robots are used to reduce manual labour and improve crop monitoring. Robots can move through crop rows, capture images, collect sensor data, and perform actions such as spraying or harvesting. In polytunnel farming, robots are useful because plants are arranged in rows and can be monitored one by one.

For cucumber plants, robotic monitoring is especially useful because the plants grow vertically. A fixed camera may miss leaves at different heights, while a mobile robot with scanning support can observe more parts of the plant. Robots can also help detect diseases earlier and reduce the need for farmers to manually inspect every plant.

However, many existing agricultural robots are designed for general field use and are not specifically suitable for low-cost cucumber polytunnel environments. Some robots detect plant diseases but do not connect the result with plant identification or spraying. Therefore, there is a need for a robot-based system that can identify plants, capture images, support AI analysis, and perform targeted spraying.

2.6. COMPUTER VISION FOR DISEASE AND STRESS DETECTION

Computer vision and deep learning are widely used for plant disease detection. Models such as CNN, YOLO, and Mask R-CNN can analyse plant images and identify disease symptoms. These systems can detect leaf colour changes, spots, mildew patterns, and damaged areas.

For cucumber cultivation, disease detection is important because downy mildew and powdery mildew can spread quickly. AI-based detection can help identify disease earlier than manual inspection. Water stress can also be detected using visual symptoms, allowing the farmer to respond quickly.

However, many disease detection systems only classify images and do not connect the result to real-world actions. For example, a model may detect powdery mildew, but the farmer still has to manually spray the plant. Some systems also do not separate fungal diseases from water stress. This is important because fungal diseases may require spraying, while water stress requires an alert or irrigation-related action. Therefore, disease detection should be connected with decision logic and targeted spraying.

2.7. TARGETED SPRAYING SYSTEMS

Traditional farming often uses blanket spraying, where the farmer sprays the whole area even if only a few plants are infected. This wastes chemicals, increases cost, and may harm the environment. Targeted spraying is a better approach because chemicals are applied only when and where they are needed.

Existing studies show that automated spraying systems can be combined with disease detection models. A camera detects the disease, and a pump or nozzle sprays the affected plant. This reduces manual labour and supports precision agriculture.

However, some targeted spraying systems do not store plant-level treatment records. They may spray a plant but not record which plant was treated. In addition, some systems do not consider disease type, confidence level, or severity. For cucumber cultivation, the system should spray only for fungal diseases such as downy mildew and powdery mildew, while water stress should generate an alert instead of spraying.

2.8. CUCUMBER DETECTION AND RIPENESS ESTIMATION

Cucumber ripeness estimation is important for harvest decision-making. Unlike some fruits, cucumbers do not show a strong colour change when they become ready for harvesting. Therefore, ripeness is usually related to physical size, such as length and diameter.

Computer vision models such as YOLO can detect cucumbers in images and generate bounding boxes. These bounding boxes can be used to estimate cucumber size. However, pixel measurements alone are not enough because the size in the image changes depending on the camera distance. Therefore, distance measurement using an ultrasonic sensor can improve ripeness estimation.

Existing cucumber detection systems mainly focus on detecting fruits for harvesting robots. Some systems do not provide a simple ripe or non-ripe decision. Therefore, a practical cucumber monitoring system should combine image-based cucumber detection with distance-assisted size estimation.

2.9. NPK MONITORING AND NUTRITION MANAGEMENT

Nutrient management is important for cucumber growth and yield. Nitrogen, phosphorus, and potassium are the main nutrients required by plants. Traditional nutrient management often depends on fixed schedules or farmer experience. However, crop nutrient needs change during different growth stages.

NPK sensors can measure soil nutrient levels and help farmers understand whether nutrient values are low, normal, or high. Automated nutrient mixing systems can also prepare nutrient solutions based on selected crop stages. These systems reduce manual work and improve nutrient control.

However, many existing nutrition systems depend on fixed schedules. They do not automatically identify the crop growth stage. In cucumber cultivation, flower detection can be used to identify the flowering stage and switch the nutrient profile. Therefore, combining NPK monitoring, automated mixing, and flower detection can improve nutrition management.

2.10. CLOUD AND MOBILE APPLICATION SUPPORT

Cloud platforms and mobile applications are widely used in smart agriculture. They allow farmers to view sensor readings, alerts, crop conditions, and system status remotely. Firebase is commonly used in prototype systems because it supports real-time updates and easy mobile application integration.

Mobile applications help farmers monitor environmental readings, disease alerts, spraying actions, ripeness status, and nutrition information. They make the system easier to use because farmers can access important information through a phone.

However, many existing mobile-based systems only display basic sensor readings. They do not combine plant identification, disease detection, spraying status, cucumber ripeness, sensor anomaly alerts, and nutrition management in one

platform. Therefore, an integrated mobile system is needed for complete cucumber polytunnel supervision.

2.11. RESEARCH GAP

The reviewed literature shows that many technologies have been used in smart agriculture. IoT sensors are used for environmental monitoring, LoRa is used for wireless communication, RFID is used for identification, robots are used for automated monitoring, AI is used for disease and fruit detection, and mobile applications are used for farmer supervision.

However, most existing systems are developed as separate modules. They do not provide a complete integrated solution for cucumber polytunnel management. Many systems monitor only the environment and do not identify individual plants. Some AI systems detect disease but do not connect the result with spraying. Some nutrition systems use fixed schedules and do not respond to real crop stages. Some mobile applications display only sensor readings and do not provide complete decision support.

Therefore, there is a need for an integrated smart cucumber polytunnel system that combines RFID-based plant identification, robotic scanning, AI-based disease and water stress detection, targeted spraying, LoRa-based monitoring, sensor anomaly detection, cucumber ripeness estimation, NPK monitoring, flower-based nutrition switching, Firebase storage, and mobile application supervision.

2.12. SUMMARY

This chapter reviewed existing studies related to smart agriculture technologies. The review showed that IoT, LoRa, RFID, robotics, computer vision, targeted spraying, cucumber detection, NPK monitoring, cloud storage, and mobile applications are useful for improving modern farming.

However, existing systems often focus on one function only. Cucumber polytunnel farming requires a more integrated system that can monitor plants, identify diseases, estimate ripeness, manage nutrients, store data, and support farmers through a mobile application. This research addresses that gap by proposing a complete smart cucumber polytunnel management system that combines these technologies into one practical prototype.

3. METHODOLOGY

3.1. OVERALL SYSTEM ARCHITECTURE

The proposed system architecture was designed as a modular and integrated smart cucumber polytunnel management platform. The main purpose of the architecture is to connect plant-level monitoring, environmental sensing, computer vision analysis, robotic actuation, nutrition control, cloud storage, and farmer supervision into one continuous workflow. Instead of developing each function as an isolated unit, the system was structured so that data collected from sensors, RFID tags, and cameras could be processed, stored, and used for decision-making.

The complete architecture consists of five main layers: the **field layer**, **sensing and acquisition layer**, **processing and decision layer**, **actuation layer**, and **cloud/mobile application layer**. The field layer represents the physical cucumber polytunnel environment, including cucumber plants, growing rows, RFID tags, irrigation lines, and nutrient delivery points. The sensing and acquisition layer collects data from the field using RFID, cameras, temperature and humidity sensors, ultrasonic sensors, and NPK sensors. The processing and decision layer analyses the collected data using AI models and rule-based decision logic. The actuation layer performs physical actions such as targeted spraying and nutrient delivery. Finally, the cloud/mobile layer stores data in Firebase and displays important outputs to the farmer through the mobile application.

The robotic module is placed at the center of the architecture because it physically moves through the cucumber rows and performs plant-level monitoring. The robot uses line-following navigation to move along the path inside the polytunnel. When it reaches a plant, the RFID reader identifies the unique RFID tag assigned to that plant. This allows the system to connect each scan, disease result, spray action, and plant status update with the correct plant record. The robot also includes a vertical scanning mechanism, where the camera moves along the cucumber plant height to capture leaf and fruit images from different levels. This is important because cucumber plants grow vertically and a single fixed camera position may not cover the complete plant structure. The individual robot thesis describes this module as a plant-level “sense–decide–act” system that combines RFID identification, vertical scanning, YOLOv8-based detection, Firebase communication, and targeted spraying.

The image processing and AI analysis module receives images captured by the robot-mounted camera. These images are used for two major tasks. The first task is cucumber leaf disease and water stress detection. In this process, the system analyses cucumber leaf images and classifies the plant condition as healthy, downy mildew, powdery mildew, or water stress. If downy mildew or powdery mildew is detected, the result is passed to the decision-making unit to activate the targeted spraying

mechanism. If water stress is detected, the system generates an alert for the farmer instead of activating fungicide spraying. The disease detection component uses robot-mounted image acquisition and is designed to support real-time plant health monitoring inside Sri Lankan polytunnel conditions.

The second image-based task is cucumber fruit detection and ripeness estimation. The system uses a YOLOv8-based object detection model to identify cucumber fruits in captured images. After a cucumber is detected, the bounding box dimensions are extracted and combined with ultrasonic sensor distance measurements to estimate the approximate physical size of the cucumber. Based on the estimated length and diameter, the system classifies the cucumber as ripe or non-ripe. This module supports harvest-readiness identification and reduces dependence on manual fruit inspection. The LoRa and cucumber monitoring component describes this integrated approach as a combination of LoRa-based sensor communication, AI-based cucumber detection, ultrasonic-assisted ripeness estimation, sensor detection, Firebase storage, and React Native mobile visualization.

The environmental monitoring module uses LoRa communication to transmit temperature and humidity readings from sensor nodes placed inside the polytunnel. These readings are collected by sensor nodes and sent wirelessly to a receiver or gateway. The gateway then uploads the readings to Firebase so that they can be viewed through the mobile application. A two-sensor anomaly detection method is also included in the architecture. In this method, readings from two temperature and humidity sensors are compared. If the difference between the two readings exceeds a defined threshold, the system identifies the reading as abnormal and generates an anomaly alert. This improves the reliability of the monitoring system because faulty or inconsistent sensor readings can be detected before they mislead the farmer.

The nutrition management module is designed as a separate but connected subsystem of overall architecture. It includes an NPK monitoring unit, an NPK mixing unit, flower detection-based season switching, and mobile application supervision. The NPK monitor measures soil nitrogen, phosphorus, and potassium values near the cucumber root zone. These readings are displayed locally using an OLED display and sent to the monitoring platform. The NPK mixer contains nutrient tanks, solenoid pumps, load cells, and a mixing barrel. Based on the selected crop growth stage, the mixer dispenses the required nutrient proportions, mixes the solution, and delivers it through the drip irrigation line. The uploaded nutrition component explains that the module integrates real-time soil nutrient monitoring, automated nutrient mixing, flower detection-based season switching, and mobile supervision into one fertigation workflow.

A major feature of the nutrition architecture is flower detection-based season switching. The robot-mounted camera captures plant images, and the flower

detection model checks whether cucumber flowers are present. If a flower is detected, the system identifies that the crop has entered the flowering stage and automatically changes the nutrient mixing profile to the flowering-season nutrient ratio. This allows the nutrition management process to respond to actual plant development instead of depending only on fixed schedules.

Firebase acts as the central cloud storage and communication layer of the system. It stores plant IDs, scan logs, disease detection results, spray decisions, environmental sensor readings, anomaly alerts, cucumber ripeness status, NPK values, nutrient mode, and device status. The React Native mobile application retrieves these data and displays them through farmer-friendly screens. Through the mobile application, the farmer can monitor tunnel conditions, plant health, cucumber ripeness, nutrition status, anomaly alerts, and system actions. This makes the system semi-automated, where automation supports the farmer while still allowing human supervision.

Overall, the system architecture follows a **sense–analyze–decide–act–record** model. First, the system senses plant and environmental conditions through sensors, RFID, and cameras. Then, the collected data are analyzed using AI models and rule-based logic. Based on the analysis, the system makes decisions such as spray or no spray, ripe or non-ripe, normal or anomalous sensor reading, and current nutrient season. After that, the system performs actions such as spraying, nutrient mixing, fertigation, alert generation, and data updating. Finally, all results are recorded in Firebase and displayed to the farmer through the mobile application.

Layer	Main Components	Function
Field layer	Cucumber plants, polytunnel rows, RFID tags, drip lines	Provides the physical cultivation environment
Sensing and acquisition layer	RFID reader, camera, NPK sensor, temperature/humidity sensors, ultrasonic sensor	Collects plant, environmental, nutrient, and fruit data
Processing and decision layer	YOLOv8, Mask R-CNN, anomaly logic, ripeness estimation logic, nutrition control logic	Analyses data and generates decisions
Actuation layer	Pump, servo nozzle, NPK mixer, solenoid pumps, drip irrigation line	Performs spraying and nutrient delivery
Cloud and mobile layer	Firebase, React Native mobile application	Stores data and displays alerts/status to the farmer

Table 3 Main architectural layers

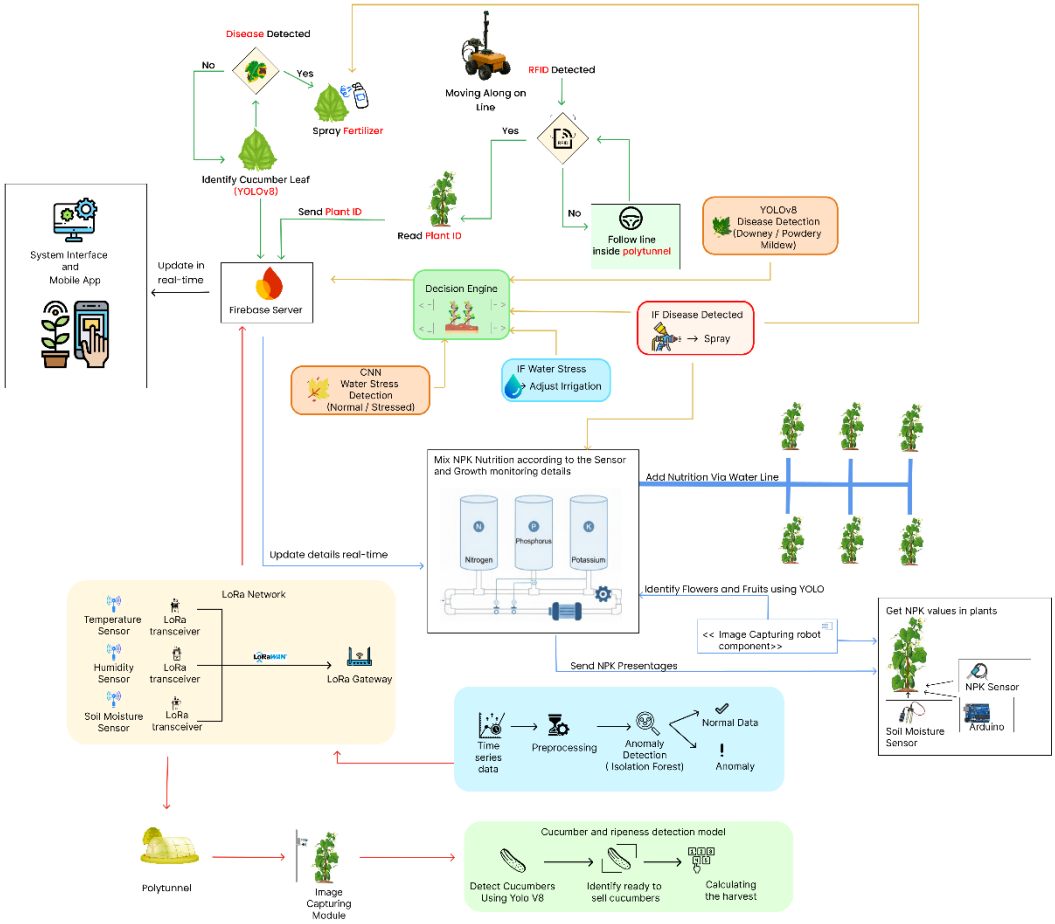


Figure 3 System Overview

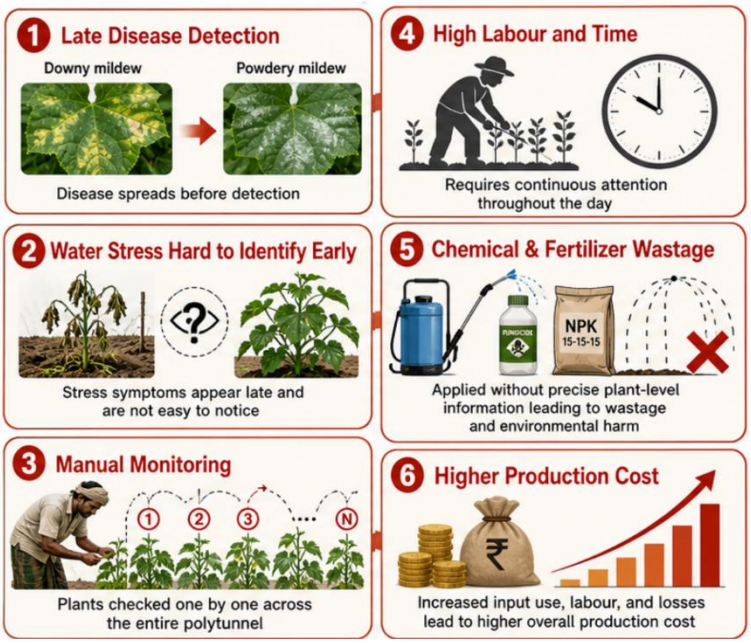


Table 4 Problem

3.2. HARDWARE ARCHITECTURE

The hardware architecture of the proposed smart cucumber polytunnel management system was designed to support physical movement, plant identification, image acquisition, environmental sensing, nutrient monitoring, automated spraying, and nutrient mixing. Since the proposed system combines several research components into one integrated platform, the hardware design follows a modular structure. Each module performs a specific function, but all modules are connected through the main control and monitoring workflow.

The main hardware units of the system are the **robot navigation unit**, **RFID plant identification unit**, **vertical image scanning unit**, **targeted spraying unit**, **LoRa-based environmental monitoring unit**, **cucumber ripeness estimation unit**, **NPK monitoring unit**, and **NPK mixing unit**. These hardware modules were selected to support the overall sense–analyze–decide–act workflow of the system.

The robot-based component uses an **ESP32 microcontroller** as the main embedded control unit. The ESP32 controls motor movement, RFID reading, vertical scanning, pump actuation, servo movement, and cloud communication. The individual robot component explains that the robot combines ESP32 control, IR-based path following, MFRC522 RFID reading, stepper-motor-based vertical scanning, mobile camera image capture, Firebase communication, and spray actuation into one working robot platform.

3.2.1. ROBOT NAVIGATION UNIT

The robot navigation unit is responsible for moving the system through the cucumber polytunnel rows. The robot platform consists of a chassis, DC motors, wheels, motor driver, IR sensors, ESP32 controller, and battery supply. The robot follows a marked path inside the polytunnel using IR sensors. This navigation method was selected because it is simple, low-cost, and suitable for a prototype-level system in a controlled polytunnel path.

When the robot moves along the cucumber row, it stops near selected plant positions for RFID reading and image capture. This movement supports plant-by-plant monitoring instead of general tunnel-level observation. Since cucumber plants are arranged in rows, the line-following robot structure is suitable for moving along the cultivation path and stopping near plant locations.

Component	Purpose
Robot chassis	Provides the physical structure of the robot
DC motors	Move the robot along the row
Motor driver	Controls the speed and direction of motors
IR sensors	Detect the line/path for navigation
ESP32 microcontroller	Controls movement and coordinates other modules
Battery supply	Provides power to the robot

Table 5 navigation unit

3.2.2. RFID PLANT IDENTIFICATION UNIT

The RFID plant identification unit is used to identify individual cucumber plants. Each selected cucumber plant is assigned an RFID tag. The robot carries an **MFRC522 RFID reader**, which reads the tag when the robot reaches the plant location. After reading the tag, the system identifies the plant and links the scan result, disease result, spray decision, and plant history to the correct plant record.

This plant-level identification is important because many basic smart farming systems only monitor environmental conditions and do not identify each plant separately. In this system, RFID helps create a digital connection between the physical plant and the cloud database. The robot thesis highlights that RFID allows scan and spray events to be saved under the correct plant record.

Component	Purpose
RFID tag	Assigned to each cucumber plant
MFRC522 RFID reader	Reads the plant tag
ESP32	Processes the RFID value
Firebase connection	Stores the plant identity and scan events

Table 6 RFID unit

3.2.3. VERTICAL IMAGE SCANNING UNIT

The vertical scanning unit was designed because cucumber plants grow upward and their leaves appear at different height levels. A fixed camera can miss important symptoms because it may capture only one part of the plant. Therefore, the proposed system uses a vertical scanning mechanism to move the camera along the plant height and capture images from multiple levels.

The vertical scanning mechanism consists of a stepper motor, motor driver, threaded shaft or vertical rail, and camera holder. A mobile phone camera or RGB camera is mounted on the vertical structure. The stepper motor moves the camera holder up and down, allowing the system to capture images from lower, middle, and upper plant regions. The robot component specifically identifies vertical scanning as a solution for tall cucumber plants and uses a mobile phone camera mounted on a vertical gantry.

Component	Purpose
Stepper motor	Moves the camera holder vertically
A4988 / stepper driver	Controls stepper motor movement
Threaded rod / vertical rail	Guides vertical movement
Camera holder	Holds the mobile phone or RGB camera
Camera	Captures leaf, flower, and cucumber images
ESP32	Controls scanning movement

Table 7 The vertical scanning unit

3.2.4. IMAGE ACQUISITION UNIT

The image acquisition unit is used to capture visual data for disease detection, water stress detection, flower detection, and cucumber fruit detection. The robot-mounted camera captures images while the robot is positioned near the plant. AI models later analyze these images to detect leaf condition, cucumber fruits, and flowers.

For disease and water stress detection, cucumber leaf images are captured and classified into healthy, downy mildew, powdery mildew, or water stress classes. The disease detection component states that the system uses a robot-mounted camera to capture cucumber leaf images inside the polytunnel and generate detection outputs for farmer decision-making.

For cucumber ripeness estimation, the camera captures fruit images. The YOLOv8 model detects cucumber fruits and produces bounding boxes. These bounding boxes are later combined with ultrasonic distance readings to estimate the real-world size of the cucumber. The LoRa and cucumber monitoring component explains that the hardware includes a camera for image capture and an ultrasonic sensor for distance measurement.

3.2.5. TARGETED SPRAYING UNIT

The targeted spraying unit is responsible for applying fungicide only when disease is detected. This unit includes a small pump, fungicide container, nozzle, servo motor, relay or motor driver, and connecting tubes. The pump is activated only when the AI decision output indicates that spraying is required. The servo motor adjusts the nozzle direction so that the spray can be directed toward the affected plant area.

This unit supports precision spraying instead of blanket spraying. If the detected class is healthy, the pump remains off. If downy mildew or powdery mildew is detected, the system activates the spraying mechanism according to the decision logic. The disease component also separates the system action by class: downy mildew triggers Mancozeb spraying, powdery mildew triggers Hexaconazole spraying, and water stress triggers a mobile alert instead of fungicide spraying.

Component	Purpose
Fungicide container	Stores selected fungicide solution
Water/pesticide pump	Pumps liquid toward the nozzle
Relay / motor driver	Controls pump activation
Servo motor	Adjusts nozzle direction
Spray nozzle	Applies fungicide to the plant
ESP32	Receives spray decision and controls actuation

Table 8 The spraying unit

3.2.6. LORA-BASED ENVIRONMENTAL MONITORING UNIT

The LoRa-based environmental monitoring unit collects temperature and humidity readings inside the cucumber polytunnel and transmits them wirelessly to the gateway. The sensor node contains temperature and humidity sensors, a

microcontroller, and a LoRa transmitter module. The receiver side contains a LoRa receiver or gateway, which receives data packets and forwards the values to Firebase.

LoRa was selected because it can transmit small environmental data packets over longer distances with low power consumption. The uploaded LoRa component describes the hardware setup as including temperature sensor, humidity sensor, LoRa transmitter module, LoRa receiver/gateway, microcontroller, camera, and ultrasonic sensor.

Component	Purpose
Temperature sensor	Measures tunnel temperature
Humidity sensor	Measures relative humidity
Microcontroller	Reads sensor values and prepares data packet
LoRa transmitter	Sends readings wirelessly
LoRa receiver / gateway	Receives transmitted packets
Firebase connection	Stores environmental readings
Mobile app	Displays values to the farmer

Table 9 The LoRa monitoring unit

3.2.7. CUCUMBER RIPENESS ESTIMATION HARDWARE

The cucumber ripeness estimation hardware uses two main inputs: image data and distance data. The camera captures cucumber images, and the YOLOv8 model identifies cucumber fruits using bounding boxes. The ultrasonic sensor measures the distance between the camera/robot and the cucumber plant. These two values are combined to estimate the real-world size of the cucumber.

This hardware design was selected because salad cucumber ripeness is related to physical size rather than strong color changes. Therefore, distance-assisted size estimation is more practical than color-only ripeness detection. The uploaded LoRa/ripeness component explains that cucumber detection uses YOLOv8 and that ultrasonic distance measurements are used to estimate harvest readiness.

Component	Purpose
Camera	Captures cucumber fruit image
Ultrasonic sensor	Measures distance from camera to cucumber
Microcontroller	Reads distance value
Processing unit	Combines bounding box size and distance

Firebase / mobile app	Stores and displays ripe or non-ripe result

Table 10 The ripeness estimation hardware

3.2.8. NPK MONITORING UNIT

The NPK monitoring unit is used to measure soil macronutrient values near the cucumber root zone. It consists of an NPK soil sensor, ESP32 microcontroller, mini-OLED display, and battery pack. The NPK sensor measures nitrogen, phosphorus, and potassium values. The ESP32 reads and processes sensor values. The OLED display shows the values locally, while the same readings can be sent to the mobile application for remote monitoring.

The nutrition component describes the NPK Monitor as a compact IoT-based device consisting of an NPK soil sensor, ESP32 microcontroller, mini-OLED display, and battery pack. It measures real-time NPK values near the cucumber root zone and supports both local and remote monitoring.

Component	**Purpose**
NPK soil sensor	Measures nitrogen, phosphorus, and potassium
ESP32	Reads and processes nutrient values
Mini OLED display	Shows values locally in the field
Battery pack	Powers the monitoring device
Mobile connection	Sends values to farmer interface

Table 11 The NPK monitoring unit

3.2.9. NPK MIXING UNIT

The NPK mixing unit prepares nutrient solutions according to the selected crop growth stage. It consists of three nutrient tubs, three solenoid pumps, three load cells, a mixing barrel, and a stirring mechanism. Each nutrient tub stores one liquid nutrient source related to nitrogen, phosphorus, or potassium. When the system selects a nutrient profile, the controller activates the required pumps. The load cells measure the dispensed quantity to improve dosing accuracy. The nutrients are then mixed inside the barrel and delivered to the cucumber plants through the drip irrigation line.

The nutrition component explains that the NPK Mixer uses three liquid nutrient tubs, three solenoid pumps, three load cells, and a mixing barrel with a stirrer. Once the decision layer selects the required nutrient ratio, the mixer dispenses, measures, mixes, mixes, and sends the solution to the drip irrigation delivery system.

Component	Purpose
Nutrient tubs	Stores nitrogen-based nutrient solution
Solenoid pumps	Dispense nutrient liquids
Load cells	Measure dispensed quantity
Mixing barrel	Holds the mixed solution
Stirrer	Mixes the nutrient solution evenly
Drip irrigation line	Delivers nutrient solution to plants

Table 12 The NPK mixing unit.

3.2.10. POWER SUPPLY AND PROTECTION

The system requires stable power for motors, sensors, microcontrollers, communication modules, pumps, and displays. Separate power lines are recommended for high-current actuators such as motors and pumps because they can introduce voltage drops or electrical noise. Low-power components such as sensors, RFID reader, OLED display, and LoRa modules should receive regulated voltage through suitable converters.

Since the system is designed for a polytunnel environment, hardware protection is important. The robot and electronic circuits should be placed inside protective enclosures to reduce exposure to humidity, water droplets, dust, fertilizer solution, and fungicide spray. The robot component also notes the need for protected electronics and controlled pump operation during testing.

Module	Hardware Components	Main Function
Robot navigation	Chassis, DC motors, wheels, motor driver, IR sensors, ESP32	Moves through cucumber rows
Plant identification	RFID tags, MFRC522 RFID reader, ESP32	Identify each plant separately
Vertical scanning	Stepper motor, driver, threaded rod/rail, camera holder	Captures images at multiple plant heights

Image acquisition	Mobile phone camera / RGB camera	Captures leaf, fruit, and flower images
Targeted spraying	Pump, fungicide tank, nozzle, servo motor, relay/driver	Applies fungicide only when needed
LoRa monitoring	Temperature sensor, humidity sensor, LoRa transmitter, LoRa receiver/gateway	Sends environmental data wirelessly
Ripeness estimation	Camera, ultrasonic sensor, microcontroller	Supports size-based cucumber ripeness estimation
NPK monitoring	NPK sensor, ESP32, OLED display, battery pack	Measures soil NPK values
NPK mixing	Nutrient tubs, solenoid pumps, load cells, mixing barrel, stirrer	Prepares stage-based nutrient solution
Delivery system	Drip irrigation line	Delivers prepared nutrient solutions for cucumber plants

Table 13 Hardware Components in the system

3.3. SOFTWARE ARCHITECTURE

The software architecture of the proposed integrated smart cucumber polytunnel management system was designed to connect embedded control, AI-based image analysis, wireless communication, cloud storage, and mobile application visualization into one coordinated workflow. Since the system includes several hardware modules, the software layer is responsible for controlling devices, processing sensor data, analyzing plant images, storing system outputs, and presenting useful information to the farmer.

The software architecture consists of five main parts: **embedded control software**, **AI model processing software**, **LoRa communication software**, **Firebase cloud database**, and **React Native mobile application**. These parts work together to support real-time monitoring and decision-making inside the cucumber polytunnel. The robot component already uses ESP32 control, React Native image capture, YOLOv8 processing, Firebase communication, and decision transfer for spraying actions. Similarly, the LoRa and cucumber monitoring components use YOLOv8, Python, Firebase, React Native, and Arduino/embedded programming for sensor monitoring and ripeness estimation.

3.3.1. EMBEDDED CONTROL SOFTWARE

The embedded control software is responsible for controlling the physical hardware modules of the system. The ESP32 microcontroller is used as the main controller for the robot and several IoT-based modules. The robot control software manages line-following navigation, RFID reading, vertical camera movement, pump activation, servo movement, and communication with Firebase. The robot follows a defined state-based workflow such as navigation, RFID detection, scanning, decision waiting, actuation, and completion.

In the robot module, the ESP32 does not directly process heavy image data. Instead, it handles lightweight control tasks and sends or receives required status values through Firebase. This design reduces the processing load on the microcontroller and allows the image processing task to be handled by a laptop or external processing unit. When the robot reaches a plant, it reads the RFID tag, updates the plant identification data, creates an image capture request, waits for the AI decision, and then activates the spraying mechanism if required.

The embedded software also controls the vertical scanning mechanism. The stepper motor is moved to predefined scanning points so that the camera can capture images from different plant heights. This is important because cucumber plants grow vertically and leaf symptoms may appear in lower, middle, or upper plant areas.

3.3.2. AI MODEL PROCESSING SOFTWARE

The AI processing software is used to analyze cucumber plant images. The system includes separate AI-based tasks for disease detection, water stress detection, cucumber fruit detection, and flower detection. Python is used as the main programming environment for AI model training, testing, and prediction processing.

For disease and water stress detection, the software analyses cucumber leaf images captured by the robot-mounted camera. The disease detection component considers healthy leaves, downy mildew, powdery mildew, and water stress as the main output classes. The methodology of the disease detection module follows an image acquisition to decision-making workflow, where robot-captured images are preprocessed and passed into the AI model for classification and action decision support.

The disease detection module also considers a hybrid model approach using YOLOv8 and Mask R-CNN. In this workflow, YOLOv8 first detects or localizes the cucumber leaf region, and the extracted region is then passed to the segmentation/classification model for disease or stress analysis. This improves the

system's ability to focus on the relevant leaf area and reduces the effect of background noise.

For cucumber detection and ripeness estimation, YOLOv8 is used to detect cucumber fruits from images. The model produces bounding boxes around detected cucumbers. The bounding box dimensions are then combined with ultrasonic distance measurements to estimate the approximate physical size of the cucumber. Based on the estimated size, the system classifies the cucumber as ripe or non-ripe.

For nutrition management, YOLOv8-based flower detection is used to identify the flowering stage of cucumber plants. When a flower is detected, the software updates the active nutrient season and changes the nutrient mixing profile to the flowering-stage configuration. The nutrition management component includes this flower-triggered switching method together with NPK monitoring, automated nutrient mixing, and mobile application supervision.

3.3.3. LORA COMMUNICATION SOFTWARE

The LoRa communication software handles wireless transmission of environmental sensor readings. Temperature and humidity sensors are connected to a microcontroller, and the readings are collected at defined time intervals. The microcontroller formats the sensor data into a packet containing values such as sensor node ID, temperature, humidity, and timestamp or packet number.

After the packet is prepared, the LoRa transmitter sends it wirelessly to the LoRa receiver or gateway. The receiver extracts the transmitted values and forwards them to Firebase. This allows the environmental data to be viewed later through the mobile application. LoRa is suitable for this system because temperature and humidity readings are small numerical data packets that do not require high-bandwidth communication.

The software also supports sensor anomaly detection. In this method, readings from two temperature and humidity sensors are compared. The system calculates the absolute difference between the two temperature values and the two humidity values. If the difference exceeds a predefined threshold, the software marks the reading as abnormal and updates the anomaly status in Firebase. This helps improve the reliability of the environmental monitoring system.

3.3.4. Firebase Cloud Database

Firebase Cloud Firestore is used as the central cloud database of the system. It stores system outputs from robots, AI models, LoRa sensor nodes, nutrition modules, and

mobile applications. Firebase acts as the communication bridge between hardware modules, AI processing units, and the farmer-facing mobile application.

The robot uses Firebase to store robot status, RFID values, plant scan logs, capture requests, AI decisions, and spray actions. The AI processing unit reads image capture requests and writes prediction outputs such as disease class, confidence value, spray decision, cucumber detection status, ripeness result, and flower detection result. The LoRa module uploads temperature, humidity, and anomaly status values. The nutrition module uploads NPK readings, nutrient mode, device status, and mixing status.

Collection / Document	**Stored Data**	**Purpose**
robots	robot ID, current RFID, movement status, scan status, spray status	Stores robot operation status
plants	plant ID, RFID value, latest disease result, latest spray result	Maintains plant-level records
scanEvents	scan ID, plant ID, image reference, detection result, timestamp	Stores plant scan history
sensorReadings	temperature, humidity, node ID, timestamp	Stores LoRa environmental data
anomalyStatus	temperature difference, humidity difference, normal/anomaly status	Stores sensor reliability status
cucumberDetections	detected/not detected, bounding box values, ripe/non-ripe status	Stores cucumber ripeness outputs
nutrition	NPK values, active season, nutrient ratio, mixer status	Stores nutrition management data
alerts	disease alerts, water stress alerts, anomaly alerts, nutrition alerts	Displays important farmer notifications

Table 14 Firebase structure

Firebase was selected because it supports cloud-based data storage and near real-time updates. This is important for the system because the farmer needs to view updated environmental readings, disease alerts, ripeness results, and nutrition information without directly accessing the hardware or backend logic.

3.3.5. REACT NATIVE MOBILE APPLICATION

The React Native mobile application is the user interface of the proposed system. It allows the farmer to view important system outputs in a simple and understandable way. The mobile application retrieves data from Firebase and displays real-time information such as robot status, plant scan results, disease detection outputs, water stress alerts, temperature and humidity values, anomaly status, cucumber ripeness result, NPK readings, active nutrient season, and mixer status.

The mobile application is also used in the image capture workflow. In the robot module, the mobile phone camera is mounted on the vertical gantry, and the React Native application captures plant images when a capture request is created. The image is then used for AI-based analysis, while the decision is returned through Firebase for robot action. This method keeps the ESP32 lightweight because the microcontroller does not directly process image data.

The nutrition module also uses the mobile application to display real-time NPK values, device status, season information, and manual override options. This is useful because the system is not designed as a fully hidden automation process. Instead, it allows the farmer to supervise and intervene when necessary.

3.3.6. SOFTWARE DATA FLOW

The software data flow starts when the robot or sensor modules collect field data. RFID values, sensor readings, images, and NPK values are collected from the physical system. The embedded software sends the required values to Firebase or triggers image capture. The AI processing software analyses the captured images and writes prediction results back to Firebase. Rule-based logic then converts the results into actions such as spray, no spray, water stress alert, ripe/non-ripe status, anomaly status, or nutrient season update.

The mobile application reads the updated Firebase values and displays them to the farmer. If the system requires physical action, such as spraying or nutrient mixing, the embedded control software reads the decision and activates the relevant actuator.

The overall software data flow can be summarized as follows:

Software Component	**Technology Used**	**Main Purpose**
Embedded programming	Arduino IDE / ESP32 programming	Controls robot movement, RFID, sensors, pumps, motors, and LoRa modules

AI model training	Python, YOLOv8, Mask R-CNN	Detects diseases, water stress, cucumbers, and flowers
Image processing	Python, OpenCV / preprocessing methods	Resizes, normalizes, enhances, and prepares images for model prediction
LoRa communication logic	Embedded C/C++	Sends and receives temperature and humidity readings
Sensor anomaly logic	Rule-based threshold comparison	Detects abnormal sensor differences
Cloud database	Firebase Firestore	Stores robot status, sensor readings, detection results, nutrition data, and alerts
Mobile application	React Native	Displays real-time system outputs to the farmer
Decision logic	Rule-based software conditions	Converts AI/sensor outputs into actions such as spray, alert, or nutrient switching

Table 15 Overall software data flow

Overall, the software architecture was designed to make the system modular, lightweight, and practical for prototype-level smart agriculture deployment. Heavy tasks such as image processing and AI prediction are handled by Python-based processing units, while real-time hardware control is handled by ESP32-based embedded software. Firebase connects all modules together, and the React Native mobile application presents the final outputs to the farmer in a clear and usable format.

3.4. DATASET PREPARATION AND DATA COLLECTION

Dataset preparation was an important stage of this research because the proposed smart cucumber polytunnel system depends on image-based analysis and sensor-based decision-making. The system uses different types of data for different modules, including cucumber leaf images for disease and water stress detection, cucumber fruit images for fruit detection and ripeness estimation, cucumber flower images for nutrition season switching, and environmental sensor readings for LoRa-based monitoring and anomaly detection. Therefore, the datasets were prepared according to the requirements of each module instead of using one general dataset for the complete system.

The main objective of the dataset preparation stage was to create suitable input data for training, validating, and testing the AI models and decision-making logic. Since the proposed system is designed for real-world polytunnel conditions, the collected data needed to represent practical challenges such as uneven lighting, overlapping leaves, different plant angles, background noise, partial occlusion, and variation in plant growth stages. These conditions are important because cucumber plants inside a polytunnel do not appear in a clean laboratory background. Therefore, preprocessing, annotation, labeling, and augmentation were applied where necessary to improve the quality and diversity of the datasets.

3.4.1. DISEASE AND WATER STRESS IMAGE DATASET

The disease and water stress detection dataset was prepared using cucumber leaf images. The purpose of this dataset was to train the model to identify whether a cucumber leaf is healthy, affected by downy mildew, affected by powdery mildew, or showing water stress symptoms. These four classes were selected because they represent the main plant health conditions considered in this research.

The images were collected from polytunnel environments and supplemented with additional labeled images to improve model generalization. Since real polytunnel images may include different lighting conditions, leaf orientations, backgrounds, and overlapping leaves, the dataset was prepared to represent these practical field conditions. The uploaded disease and water stress component states that the dataset contains four classes: healthy, downy mildew, powdery mildew, and water stress, with a total of 1,970 cucumber leaf images.

The collected images were organized according to their class labels. Before model training, preprocessing techniques were applied to improve image consistency. These preprocessing steps included image resizing, normalization, image enhancement, and data augmentation. Augmentation was used to increase dataset diversity and reduce overfitting by exposing the model to different visual variations. The disease detection draft mentions resizing, normalization, image enhancement, and data augmentation as dataset preparation steps used to improve dataset quality and diversity.

The final prepared dataset was divided into training, validation, and testing sets. The training set was used to train the model, the validation set was used to tune and monitor performance during training, and the testing set was used to evaluate the

final model performance. This separation helped ensure that the model was evaluated using images that were not directly used during training.



Figure 6 Healthy Leaf



Figure 5 Downey Mildew



Figure 4 Powdery Mildew



Figure 7 Water Stress

3.4.2. CUCUMBER FRUIT DETECTION DATASET

The cucumber fruit detection dataset was prepared to train the YOLOv8 object detection model for identifying cucumber fruits from polytunnel images. This dataset was required because cucumber fruits are difficult to detect in real polytunnel conditions due to their green color, which is like leaves and stems. In addition, fruit occlusion, shadows, different fruit sizes, and complex backgrounds can reduce detection accuracy.

The cucumber detection dataset consisted of 891 cucumber images. These images were prepared and annotated for object detection training. Each cucumber fruit in the images was marked using bounding boxes so that the YOLOv8 model could learn the location and shape of cucumber fruits. The uploaded cucumber detection component states that the YOLOv8 model was trained using 891 cucumber images and that the detected bounding boxes were used to obtain image-based cucumber length and width for ripeness estimation.

The annotation process produced bounding box coordinates for each cucumber fruit. These bounding boxes were important for two reasons. First, they allowed the model to detect the presence and location of cucumber fruits. Second, the bounding box width and height were later used by the ripeness estimation module to calculate pixel-based cucumber dimensions. These pixel measurements were combined with ultrasonic distance readings to estimate the approximate real-world size of the cucumber.



Figure 8 Cucumber Fruit

3.4.3. FLOWER DETECTION DATASET

The flower detection dataset was prepared for the nutrition management module. In the proposed system, flower detection is used to identify whether the cucumber crop has reached the flowering stage. When flowers are detected, the system automatically switches the nutrient profile to the flowering-season nutrient combination.

The flower detection module was developed using YOLOv8. The dataset used for this task consisted of 798 cucumber plant images, and visible cucumber flowers were labeled using bounding boxes through the Roboflow platform. The uploaded nutrition management component states that the flower detection dataset consisted of 798 cucumber plant images and that preprocessing and augmentation methods such as resizing, normalization, rotation, flipping, brightness adjustment, and zooming were used to improve detection robustness under practical polytunnel conditions.

Flower detection required careful dataset preparation because cucumber flowers are small and may be partially hidden by leaves. They can also appear under different lighting conditions and viewing angles. Therefore, augmentation was useful for improving the model's ability to detect flowers under different practical conditions.



Figure 9 Cucumber Flower

3.4.4. ENVIRONMENTAL SENSOR DATA

Environmental sensor data were used for the LoRa-based monitoring and anomaly detection module. The system collected temperature and humidity readings from two sensor nodes placed inside the polytunnel. These readings were transmitted using LoRa communication and uploaded to Firebase through the receiver or gateway.

Unlike the image datasets, environmental data were numerical values rather than images. Each sensor data record included the sensor node identifier, temperature value, humidity value, and timestamp or packet number. The LoRa monitoring methodology explains that the microcontroller reads the sensor values, arranges them into a packet, transmits the packet using LoRa, and then uploads the received values to Firebase for mobile application display.

The same data were also used for anomaly detection. The anomaly detection method compared the readings from two sensors. If the difference between the two temperature values or humidity values exceeded the defined threshold, the reading was marked as abnormal. This helped the system identify sensor faults, unstable readings, or environmental measurement errors.

3.4.5. NPK AND NUTRITION DATA

The nutrition management module used soil NPK readings and nutrient profile data. The NPK monitoring device measured nitrogen, phosphorus, and potassium values near the cucumber root zone. These values were displayed locally using an OLED display and sent to the monitoring platform for mobile application viewing.

The NPK mixer used predefined nutrient profiles for different crop growth stages. These nutrient profiles controlled the amount of nitrogen, phosphorus, and potassium solution dispensed by the mixing unit. The flower detection output was also connected to this dataset because the detection of flowers triggered the system to switch to the flowering-stage nutrient profile. The nutrition management component explains that the system includes an NPK Monitor, NPK Mixer, and image-based flower detection mechanism to support automated nutrient-stage switching.

3.4.6. DATASET PREPROCESSING AND AUGMENTATION

Preprocessing was applied to make image datasets more suitable for model training. Since the images were collected under practical polytunnel conditions, they could contain noise, lighting variation, background objects, shadows, and different image sizes. Therefore, preprocessing helped standardize the inputs before they were passed into the AI models.

For object detection datasets such as cucumber fruit detection and flower detection, annotation was performed using bounding boxes. For disease and water stress detection, images were organized according to class labels. In the hybrid disease detection approach, leaf localization and segmentation were also considered, so the dataset preparation process supported both classification and localization-based model development.

3.4.7. SUMMARY OF PREPARED DATASETS

The prepared datasets supported the main AI and decision-making modules of the proposed system. The disease and water stress dataset was used to classify cucumber leaf health conditions. The cucumber fruit dataset was used to train a YOLOv8 model for fruit detection and ripeness estimation. The flower detection dataset was used to identify the flowering stage and trigger nutrition season switching. Environmental sensor data were used for LoRa monitoring and anomaly detection, while NPK and nutrient profile data were used for automated nutrition management.

Overall, the dataset preparation stage provided the foundation for the intelligent functions of the proposed smart cucumber polytunnel system. Properly prepared datasets improved the reliability of disease detection, cucumber detection, flower detection, ripeness estimation, anomaly detection, and nutrition management.

3.5. AI MODEL DEVELOPMENT METHODOLOGY

The AI model development methodology describes how computer vision was used to analyze cucumber plant images and generate useful decisions for the proposed smart polytunnel system. Since the complete system includes plant disease detection, water stress detection, cucumber fruit detection, ripeness estimation, and flower-based nutrition season switching, different AI workflows were developed according to the purpose of each module.

The AI component uses **YOLOv8-based object detection** and a **hybrid disease detection approach**. YOLOv8 was used because it is suitable for detecting objects in images and producing bounding boxes around important plant features such as cucumber leaves, cucumber fruits, and cucumber flowers. For disease and water stress detection, a hybrid model was considered because the system needs not only to classify the plant condition but also to identify the affected region and estimate disease severity. The disease detection component explains that two approaches were considered: a direct classification model and a hybrid model combining YOLOv8 and Mask R-CNN, where the hybrid model was selected because it provided better accuracy and more useful localization and severity outputs.

3.5.1. DISEASE AND WATER STRESS DETECTION MODEL

The disease and water stress detection model was developed to identify the health condition of cucumber leaves captured by the robot-mounted camera. The model classifies the leaf condition into four main classes:

Output Class	Meaning	System Action
Healthy	No visible disease or water stress symptoms	No action
Downy mildew	Downy mildew disease symptoms detected	Activate targeted fungicide spraying
Powdery mildew	Powdery mildew disease symptoms detected	Activate targeted fungicide spraying
Water stress	Water stress symptoms detected	Send alert to farmer

Table 16 Main Classes of the model classify into

The image processing workflow begins when the robot captures a cucumber leaf image during vertical scanning. The image is then passed to the AI processing pipeline. In the hybrid model, YOLOv8 first detects the cucumber leaf region from the image. This step is important because robot-captured images may contain background objects such as stems, ropes, grow bags, pipes, soil, and overlapping leaves. By detecting the leaf area first, the system reduces background interference and focuses only on the important plant region.

After the leaf region is detected, the cropped or extracted leaf area is passed to the Mask R-CNN model. Mask R-CNN performs classification and segmentation. The classification output identifies whether the leaf is healthy, affected by downy mildew, affected by powdery mildew, or showing water stress. The segmentation output identifies the affected area on the leaf. This is useful because it allows the system to estimate disease severity instead of only giving a class label.

The severity estimation is calculated by comparing the affected leaf area with the total detected leaf area. The disease component defines this severity concept using the affected leaf pixels and total leaf pixels, which helps the system decide whether the disease is at a low, medium, or high level.

Disease Severity (%) = (Affected Leaf Area / Total Leaf Area) × 100

Severity Level	Severity Percentage	Recommended Action
Low	1%–25%	Monitor and notify
Medium	26%–50%	Targeted fungicide spraying

High	Above 50%	Immediate spraying and farmer alert

Table 17 Severity Levels

This severity-based method supports more controlled chemical application. For example, early-stage symptoms can be recorded and monitored, while medium or high severity disease can trigger spraying. This helps reduce unnecessary blanket spraying and supports precision agriculture.

3.5.2. DISEASE AND STRESS DECISION LOGIC

After the disease and water stress model produces the prediction result, the decision logic converts the output into a system action. If the detected class is healthy, no spraying action is performed. If downy mildew is detected, the system activates the spraying unit with the selected fungicide for downy mildew. If powdery mildew is detected, the system activates the spraying unit with the selected fungicide for powdery mildew. If water stress is detected, the system does not activate the fungicide pump because water stress is not a fungal disease. Instead, an alert is sent to the farmer through the mobile application.

AI Detection Result	**Decision**	**System Response**
Healthy	No disease or stress detected	Continue scanning
Downy mildew	Disease detected	Spray downy mildew fungicide
Powdery mildew	Disease detected	Spray powdery mildew fungicide
Water stress	Stress detected	Send mobile alert
Low confidence prediction	Uncertain result	Store for review / avoid automatic action

Table 18 AI Detection Results

The decision output is sent to Firebase so that the robot can read the result. If the output is SPRAY, the robot activates the pump and nozzle system. If the output is NO_SPRAY, the robot continues scanning or moves to the next plant. This connects image-based AI detection with physical robotic action.

3.5.3. CUCUMBER FRUIT DETECTION MODEL

The cucumber fruit detection model was developed to identify cucumber fruits from images captured inside the polytunnel. A YOLOv8 object detection model was used for this task. The model detects cucumber fruits and produces bounding boxes

around them. These bounding boxes are important because they provide the pixel-based width and height of the detected cucumber.

The cucumber detection component states that the YOLOv8 model was trained using **891 cucumber images**, and the model output bounding boxes were later used for cucumber size estimation and ripeness classification.

The cucumber detection workflow includes the following steps:

1. Capture cucumber plant image using the camera.
2. Pass the image into the trained YOLOv8 cucumber detection model.
3. Detect cucumber fruit in the image.
4. Generate bounding box coordinates around the cucumber.
5. Extract pixel-based width and height from the bounding box.
6. Send extracted values to the ripeness estimation module.

Output	**Description**
Detection class	Cucumber
Confidence score	Model confidence in the detected cucumber
Bounding box x-coordinate	Horizontal location of detected cucumber
Bounding box y-coordinate	Vertical location of detected cucumber
Bounding box width	Pixel-based cucumber width
Bounding box height	Pixel-based cucumber height

Table 19 Model Output

3.5.4. CUCUMBER RIPENESS ESTIMATION METHOD

In this research, cucumber ripeness refers to **harvest readiness** based on physical size rather than color. This is important because salad cucumbers usually remain green even when they are ready for harvesting. Therefore, color-based ripeness estimation is less suitable. Instead, the system uses size-based ripeness estimation.

After the cucumber is detected by YOLOv8, the bounding box dimensions are extracted. However, pixel values alone cannot directly represent the real-world size of the cucumber because the apparent size changes depending on the camera

distance. To solve this problem, an ultrasonic sensor is used to measure the distance between the camera and the cucumber plant. The bounding box dimensions and ultrasonic distance value are then combined to estimate the approximate real-world length and diameter of the cucumber.

The cucumber ripeness component explains that the system combines YOLOv8 bounding box dimensions with ultrasonic distance measurements, then compares the estimated cucumber length and diameter with the selected ripe size range.

Estimated Measurement	**Ripe Condition**
Length	Around 15–16 cm
Diameter	Around 2.5–3.0 cm
Final output	Ripe / non-ripe

Table 20 Ripeness Detection

If the estimated length and diameter fall within the selected ripe range, the cucumber is classified as ripe. If the cucumber is smaller or larger than the selected range, it is classified as non-ripe. The result is stored in Firebase and displayed in the mobile application.

3.5.5. FLOWER DETECTION MODEL FOR NUTRITION SEASON SWITCHING

The flower detection model was developed to support the nutrition management module. In cucumber cultivation, flowering is an important growth-stage indicator because nutrient demand changes when the plant moves from vegetative growth to flowering and fruit development. Therefore, flower detection is used as a biological trigger to switch the nutrient profile.

The flower detection model was developed using YOLOv8. The nutrition management component states that the flower detection dataset consisted of **798 cucumber plant images**, where visible flowers were labeled using bounding boxes through Roboflow. Preprocessing and augmentation techniques such as resizing, normalization, rotation, flipping, brightness adjustment, and zooming were applied to improve robustness under practical polytunnel conditions.

The flower detection workflow includes the following steps:

1. Capture cucumber plant images during robot scanning.
2. Pass the image into the trained YOLOv8 flower detection model.

3. Detect visible cucumber flowers using bounding boxes.
4. Check the confidence score of the detected flower.
5. Accept the detection if the confidence score is above the selected threshold.
6. Set flower status as detected.
7. Switch the active nutrition profile to the flowering-stage nutrient profile.

decision logic to include:

If flower detected = Yes:
  Active Season = Flowering Season
  Use flowering-stage NPK ratio
Else:
  Continue current nutrient schedule

Stage	Input	Processing	Output
Image capture	Plant image	Camera captures plants during scanning	Captured image
Flower detection	Captured image	YOLOv8 detects flower bounding boxes	Flower detected / not detected
Confidence filtering	Detection confidence	Remove weak detections	Valid flower detection
Season decision	Valid flower result	Check flower presence	Active season updated
Nutrition control	Active season	Select NPK ratio	Updated nutrient profile

Table 21 Flower Detection Stages

This method allows the nutrition system to respond to actual plant development instead of only depending on fixed dates. If flowering begins earlier or later than expected, the system can still change the nutrient profile based on real visual evidence.

3.5.6. MODEL TRAINING AND EVALUATION

Each AI model was trained using its prepared dataset. The datasets were divided into training, validation, and testing subsets. The training set was used to teach the model, the validation set was used to monitor training performance and reduce overfitting, and the test set was used to evaluate final model performance.

The disease detection component uses standard evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrix to compare model performance. The same type of evaluation was used to understand how well the model performs under real polytunnel conditions.

Metric	Purpose
Accuracy	Measures overall correct predictions
Precision	Measures of how many predicted positive detections are correct
Recall	Measures of how many actual target cases are detected
F1-score	Balances precision and recall
Confusion matrix	Shows class-wise misclassification
mAP@0.5	Measures object detection accuracy at IoU 0.5
mAP@0.5:0.95	Measures detection accuracy across stricter IoU thresholds

Table 22 Model Training Metrices

For disease and stress detection, accuracy, precision, recall, F1-score, and confusion matrix are important. For cucumber fruit detection and flower detection, object detection metrics such as precision, recall, mAP@0.5, and mAP@0.5:0.95 are important because the model must both detect and localize the object correctly.

3.5.7. INTEGRATION OF AI MODELS WITH THE COMPLETE SYSTEM

After training and testing, the AI models were integrated with the overall smart polytunnel system. The robot captures images during scanning and sends the image for AI processing. The AI output is then converted into a decision and stored in Firebase. The robot and mobile application read the updated result from Firebase.

The integration process follows this flow:

- Robot Image Capture
- AI Model Processing
- Prediction Result
- Decision Logic
- Firebase Update
- Robot Action / Mobile Alert

For disease detection, the output controls spraying or alert generation. For cucumber detection, the output supports ripeness estimation. For flower detection, the output

controls nutrition season switching. Therefore, the AI models act as the main decision-support layer of the proposed smart cucumber polytunnel system.

Overall, the AI model development methodology provides the intelligence layer of the proposed system. It allows the system to detect plant health problems, identify cucumber fruits, estimate harvest readiness, detect flowering stage, and convert these results into useful decisions for spraying, nutrition control, and farmer alerts.

3.6. ROBOT OPERATION WORKFLOW

The robot operation workflow describes how the proposed smart cucumber polytunnel system performs plant-level monitoring and targeted spraying. The robot was designed to move along cucumber rows, identify individual plants, capture images through vertical scanning, receive AI-based decisions, and perform spraying only when required. This workflow connects physical movement, plant identification, image acquisition, AI analysis, Firebase communication, and actuation into one complete process.

The robot follows a **sense–decide–act** workflow. First, it moves along a marked path inside the polytunnel using IR sensors. When the robot reaches a cucumber plant, the RFID reader reads the plant's RFID tag and identifies the plant. After identification, the robot stops at predefined positions near the plant and starts the vertical scanning sequence. During scanning, the camera captures cucumber leaf images at different height levels. These images are analyzed by the AI model, and the result is sent back to the robot through Firebase. If the result indicates downy mildew or powdery mildew, the spraying mechanism is activated. If the plant is healthy, the robot continues to the next plant. If water stress is detected, the system sends an alert to the farmer instead of spraying fungicide.

The robot thesis describes the expected robot workflow as: follow the polytunnel path, detect the RFID tag, stop at plant positions A, B, and C, move the camera vertically using the stepper gantry, capture leaf images, process images using the YOLOv8 model, receive a SPRAY or NO_SPRAY decision, activate the pump and servo nozzle if spraying is required, and store robot status and scan events in Firebase.

3.6.1. LINE-FOLLOWING NAVIGATION

The first stage of the robot operation is navigation. The robot moves along a marked path inside the cucumber polytunnel using two IR sensors. These sensors detect the line and guide the robot to move forward, turn left, turn right, or stop. This method

was selected because cucumber polytunnel paths are usually structured and predictable, making line-following suitable for a low-cost prototype.

The main navigation functions are:

Function	Purpose
forward()	Moves the robot forward along the row
leftTurn()	Corrects the robot when it moves away from the path
rightTurn()	Correct the robot in the opposite direction
stopMotors()	Stops the robot before RFID reading, scanning, or spraying

Table 23 Robot Line Following Functions

The robot moves forward when both IR sensors detect the correct path condition. If the robot deviates from the path, the motor control logic turns the robot left or right until it returns to the path. When RFID is detected or a plant position marker is reached, the robot stops and begins the next operation stage. The individual robot component explains that the robot uses IR sensors for path following and stops when RFID or plant position markers are detected.

3.6.2. RFID-BASED PLANT IDENTIFICATION

After reaching the plant location, the robot reads the RFID tag placed near the cucumber plant. Each plant is assigned a unique RFID tag, and the robot uses the MFRC522 RFID reader to detect the tag value. This RFID value is converted into a readable string and used to identify the plant.

RFID is important because it allows the system to link each scan result, disease result, spray action, and plant history to the correct cucumber plant. Without RFID, the system may capture images, but it cannot accurately store which plant was scanned or treated. The robot component explains that RFID supports plant-level tracking by storing scan results, disease results, and spraying actions under the correct plant record. It also maps RFID values to plant records using Firebase through the path tunnels/{assignedTunnelId}/rfidMap/{RFID}.

Step	Description	Output
1	Robot reaches plant area	Robot stops

2	MFRC522 reader scans RFID tag	RFID UID detected
3	RFID UID is converted into text	RFID string generated
4	Firebase mapping is checked	Plant ID identified
5	Plant scan session is created	Scan linked to correct plant

Table 24 RFID-BASED PLANT IDENTIFICATION

3.6.3. POSITION-BASED PLANT SCANNING

After RFID identification, the robot scans the plant at predefined positions. The system uses plant positions such as **A, B, and C** to make the scanning process more structured. This allows the robot to scan different sides or points of the same plant instead of capturing images randomly.

The position-based workflow improves plant coverage and makes the scan logs more meaningful. For example, if a disease is detected at position B, the system can store that information under the same plant ID with the exact scan position. This supports better plant-level monitoring and future traceability.

The robot component states that after RFID detection, the robot detects position markers for each plant and uses labels A, B, and C to scan the plant at multiple points. This ensures that each plant is scanned at defined positions and helps Firebase logs store the current plant position.

3.6.4. VERTICAL GANTRY SCANNING

Cucumber plants grow vertically, and their leaves appear at different height levels. Therefore, a fixed camera position is not sufficient to inspect the full plant. To solve this, the robot uses a vertical gantry mechanism to move the camera up and down along the cucumber plant.

The vertical gantry is controlled by a stepper motor. At each scan level, the camera captures an image of the plant. These images are later used for disease detection, water stress detection, flower detection, or cucumber fruit detection. Vertical scanning improves image coverage because it captures lower, middle, and upper plant regions.

The robot component explains that the vertical gantry uses a stepper motor to move the camera along the cucumber plant, and at each scan stop, the robot creates a capture request for image acquisition.

Stage	Robot Action	Purpose
Start scan	Robot stops near plant	Stabilize image capture
Move camera upward	Stepper motor moves camera	Capture different plant levels
Capture image	Mobile camera captures plant image	Collect leaf/fruit/flower data
Send capture request	Firebase updated	Trigger AI processing
Continuing scan	Camera moves to next level	Complete plant coverage
End scan	Camera returns or stops	Prepare for next action

Table 25 VERTICAL GANTRY SCANNING

3.6.5. IMAGE CAPTURE AND FIREBASE COMMUNICATION

During the scanning process, the robot creates an image capture request. The mobile camera or robot-mounted camera captures the image and sends it for AI processing. Firebase acts as the communication bridge between the robot, image capture unit, AI processing unit, and mobile application.

The robot updates Firebase with details such as robot ID, current plant ID, RFID value, scan position, capture request status, and scan event. The AI processing module reads the image or capture request, performs prediction, and writes the decision back to Firebase. The robot then reads the decision and performs the next action.

A simplified Firebase-based communication flow is:

- Robot reaches plant.
- RFID detected.
- Plant ID loaded from Firebase.
- Robot creates image capture request.
- Camera captures image
- AI model analyses image
- Decision written to Firebase.
- Robot reads decision.
- Spray / No spray / Alert action.

This cloud-based workflow allows the ESP32 robot controller to remain lightweight because heavy AI processing is managed separately.

3.6.6. AI DECISION INTEGRATION

After the image processed, the AI module generates a plant condition result. The disease and water stress detection module produces four outputs: healthy, downy mildew, powdery mildew, and water stress. Each output relates to a system action. Healthy leaves do not require action, downy mildew triggers Mancozeb spraying, powdery mildew triggers Hexaconazole spraying, and water stress sends an alert to the farmer through the mobile application.

AI Output	**Robot / System Decision**	**Action**
Healthy	NO_SPRAY	Robot continues scanning or moves forward
Downy mildew	SPRAY	Activate fungicide spraying
Powdery mildew	SPRAY	Activate fungicide spraying
Water stress	ALERT	Send mobile alert, no fungicide spraying
Low confidence	REVIEW	Store result for farmer review

Table 26 AI decisions

This decision-making process prevents unnecessary chemical application. Fungicide applied only when a fungal disease detected. Water stress handled separately because it requires irrigation attention, not fungicide spraying.

3.6.7. TARGETED SPRAYING OPERATION

The targeted spraying operation starts only after the robot receives a valid SPRAY decision. The spraying system consists of a pump, fungicide container, tube, nozzle, and servo motor. The pump pushes the fungicide solution through the nozzle, while the servo motor adjusts the spraying direction.

If the AI decision is NO_SPRAY, the pump remains off. If the AI decision delayed or not received, the robot follows a safety-based fallback and avoids spraying. This is important because accidental spraying can waste fungicide and expose the crop or farmer to unnecessary chemical use. The robot thesis states that safety methods include keeping the pump off unless the decision is SPRAY, selecting NO_SPRAY if no decision received, stopping before scanning, controlling spray duration, and recommending manual supervision during testing.

Condition	**System Response**
Valid SPRAY decision received	Activate pump and nozzle
NO_SPRAY decision received	Keep pump off
Water stress detected	Send alert, keep pump off
Decision timeout	Default to NO_SPRAY
Robot not stopped properly	Do not spray
Scan incomplete	Do not spray

Table 27 Spray Methods

3.6.8. ROBOT STATE-BASED WORKFLOW

The robot operation designed using a state-based workflow. This makes the system easier to control because each stage has a clear purpose. The main states include navigation, scanning, actuation, and completion. The robot thesis mentions that the robot code supports the workflow using states such as LINE_FOLLOW, STEPPER_SCAN, ACTUATION, and FINISHED, together with RFID detection, Firebase writing, capture request handling, timeout fail-safe, pump control, and servo nozzle movement.

State	**Description**	**Main Output**
LINE_FOLLOW	Robot follows marked path	Reaches plant area
RFID_DETECT	RFID tag read	Plant ID identified
STEPPER_SCAN	Camera moves vertically	Plant images captured
AI_WAIT	Robot waits for AI result	Decision received

ACTUATION	Spraying action performed if required	Spray/no spray completed
LOG_UPDATE	Firebase records updated	Scan event stored
FINISHED	Robot completes current plant task	Move to next plant

Table 28 Robot Statuses

3.6.9. COMPLETE ROBOT OPERATION FLOW

The complete robot operation workflow shown below:

- Robot follows marked path.
- RFID tag detected
- Robot stops near plant.
- Plant ID loaded from Firebase.
- Robot scans positions A, B, and C
- Vertical Gentry moves camera.
- Camera captures plant images
- AI model analyses image
- Decision sent to Firebase
- Robot reads decision.
- Is disease detected?
  ├── Yes → Activate targeted spraying
  └── No → Continue movement
- Is water stress detected?
  ├── Yes → Send mobile alert
  └── No → Continue normal process
- Store scan and action records in Firebase.
- Move to next plant.

Overall, the robot operation workflow provides the physical automation layer of the proposed smart cucumber polytunnel system. By combining line-following navigation, RFID-based plant identification, position-based stopping, vertical camera scanning, AI-based decision-making, targeted spraying, and Firebase event logging, the robot supports accurate plant-level monitoring and precision treatment inside the cucumber polytunnel.

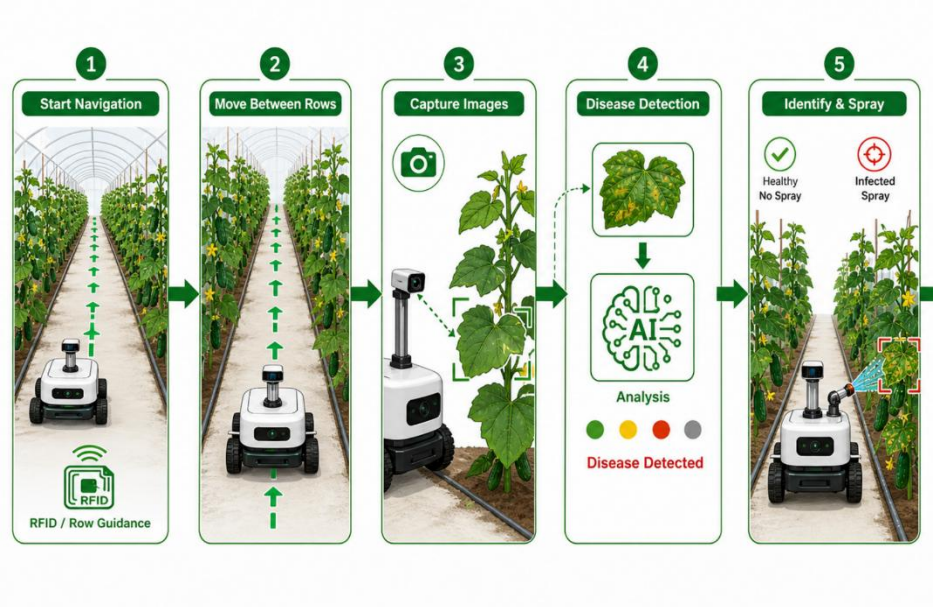


Figure 10 Robot Operation Flow

3.7. DISEASE DETECTION AND SPRAYING DECISION LOGIC

This section explains how the proposed system converts AI-based plant health detection results into practical spraying and alert actions. The disease and spraying decision logic is an important part of the integrated smart cucumber polytunnel system because it connects the image analysis module with the physical actuation system of the robot. Instead of spraying all plants manually, the system first analyses the cucumber leaf condition and then decides whether spraying is required.

The disease detection module focuses on four main cucumber leaf conditions: **healthy**, **downy mildew**, **powdery mildew**, and **water stress**. These conditions were selected because downy mildew and powdery mildew are common fungal diseases in cucumber cultivation, while water stress is also an important plant health issue that affects growth and yield. The uploaded disease detection component defines these four output classes and links them to system actions such as no action, fungicide spraying, or mobile alert generation.

The robot-mounted camera captures cucumber leaf images during the vertical scanning process. These images are analyzed by the AI model, and the prediction result is sent to the decision-making unit. The decision-making unit then checks the detected class, confidence score, and disease severity level before activating any action. This prevents the system from spraying unnecessarily when the detection result is uncertain or when the plant is healthy.

3.7.1. DISEASE DETECTION OUTPUT CLASSES

The AI model produces a plant health classification result after analyzing the captured cucumber leaf image. Each output class has a different meaning and requires a different system response.

Output Class	Meaning	System Action
Healthy	No visible disease or water stress symptoms	No spraying action
Downy mildew	Downy mildew symptoms detected	Activate targeted fungicide spraying
Powdery mildew	Powdery mildew symptoms detected	Activate targeted fungicide spraying
Water stress	Water stress symptoms detected	Send mobile alert to farmer
Low confidence result	Prediction is uncertain	Store results for review and avoid automatic spraying

Table 29 DISEASE DETECTION OUTPUT CLASSES

The system separates fungal disease detection from water stress detection. This is important because downy mildew and powdery mildew require fungicide treatment, but water stress requires irrigation attention or further farmer inspection. Therefore, the robot does not spray fungicide when water stress detected.

3.7.2. CONFIDENCE-BASED DECISION FILTERING

The AI model output includes a confidence score. The confidence score indicates how certain the model is about the detected class. If the confidence score is high, the result is accepted by the decision logic. If the confidence score is low, the system avoids automatic spraying and marks the result for review.

This step is important for safety because wrong detection may lead to unnecessary chemical spraying. For example, if the model is not confident whether a leaf has disease or water stress, the system should not immediately activate the spray pump. Instead, the result can stored in Firebase and displayed to the farmer for manual checking.

Confidence Level	Decision
High confidence	Accept prediction and continue decision process

Medium confidence	Store results and notify farmers if needed
Low confidence	Avoid automatic spraying and mark for review

Table 30 CONFIDENCE-BASED DECISION FILTERING

3.7.3. DISEASE SEVERITY ESTIMATION

After disease classification, the system also considers the severity level of the detected disease. Severity estimation helps identify how much of the leaf area is affected. Instead of only saying "disease detected," the system estimates whether the infection is low, medium, or high. The uploaded disease component explains that severity is calculated by comparing the affected leaf area with the total detected leaf area, using segmentation output from the hybrid model.

Severity Level	**Severity Percentage**	**Recommended Action**
Low	1%–25%	Monitor and notify
Medium	26%–50%	Targeted fungicide spraying
High	Above 50%	Immediate spraying and farmer alert

Table 31 DISEASE SEVERITY ESTIMATION

Severity estimation reduces unnecessary chemical usage. If the affected area is very small, the system can notify the farmer and continue monitoring. If the disease severity is medium or high, the robot performs targeted spraying.

3.7.4. FUNGICIDE SELECTION LOGIC

The spraying decision depends on the type detected disease. Since downy mildew and powdery mildew are different fungal diseases, the system uses separate treatment decisions for each disease condition. In the proposed logic, **Mancozeb** is assigned for downy mildew, while **Hexaconazole** assigned for powdery mildew. The disease detection draft specifically links downy mildew to Mancozeb spraying and powdery mildew to Hexaconazole spraying through the robotic system.

Detected Condition	**Fungicide Decision**	**System Response**
Downy mildew	Mancozeb	Activate sprayer with downy mildew treatment
Powdery mildew	Hexaconazole	Activate sprayer with powdery mildew treatment

Water stress	No fungicide	Send irrigation/stress alert
Healthy	No fungicide	Continue scanning

Table 32 FUNGICIDE SELECTION LOGIC

This logic ensures that the system does not treat all plant problems in the same way. Disease conditions trigger fungicide spraying, while water stress triggers a farmer alert.

3.7.5. SPRAYING ACTIVATION LOGIC

The robot activates the spraying mechanism only after receiving a valid SPRAY decision. The spraying unit consists of a pump, fungicide container, tube, nozzle, and servo-controlled direction mechanism. The pump remains off by default and turned on only when the decision logic confirms that spraying required.

The robot component identifies targeted spraying as a major improvement over blanket spraying because it applies treatment only when needed. It also states that the proposed robot connects RFID identification, vertical scanning, YOLOv8-based detection, Firebase communication, and targeted spraying into a plant-level “sense–decide–act” system.

spraying logic:

If detected class = Downy mildew AND severity = Medium or High:
  Decision = SPRAY
  Fungicide = Mancozeb

Else if detected class = Powdery mildew AND severity = Medium or High:
  Decision = SPRAY
  Fungicide = Hexaconazole

Else if detected class = Water stress:
  Decision = ALERT
  Send mobile notification

Else:
  Decision = NO_SPRAY
  Continue scanning

3.7.6. SAFETY AND FAIL-SAFE LOGIC

Safety is important because robots control chemical spraying. Therefore, the system uses fail-safe rules to avoid unnecessary or accidental spraying. The pump remains off unless the final decision clearly marked as SPRAY. If no decision received, the robot selects NO_SPRAY by default. The robot also stops before spraying so that the nozzle can target the plant properly. The individual robot thesis includes safety rules such as keeping the pump off unless the decision is SPRAY, using NO_SPRAY when no decision received, stopping before scanning, controlling spray duration, protecting electronics, and recommending manual supervision during testing.

Condition	Safety Response
No AI decision received	Default to NO_SPRAY
Low confidence prediction	Do not spray automatically
Robot still moving	Do not spray
Healthy class detected	Keep pump off
Water stress detected	Keep pump off and send alert
Spray duration completed	Turn pumps off
Emergency/manual stop triggered	Stop motors and pump

Table 33 SAFETY AND FAIL-SAFE LOGIC

These safety rules required because the system is designed for a real cultivation environment where accidental spraying can waste fungicide, damage plants, or create safety risks for farmers.

3.7.7. FIREBASE-BASED TRANSFER

Firebase is used to transfer the AI decision from the image processing module to the robot. After the AI model analyses the plant image, the result is stored in Firebase. The robot then reads the decision field and performs the correct action. This design keeps the ESP32 robot controller lightweight because it does not need to run the AI model directly.

Field Name	Example Value	Purpose
plantId	PLANT_001	Identifies the scanned plant

rfid	A1B2C3D4	Stores RFID tag value
detectedClass	Downy Mildew	Stores AI prediction
confidence	0.94	Stores confidence score
severity	Medium	Stores severity level
decision	SPRAY	Final system decision
fungicide	Mancozeb	Selected treatment
sprayDurationMs	3000	Pump activation duration
timestamp	Date/time	Stores event time

Table 34 FIREBASE-BASED DECISION TRANSFER

This structure supports plant-level traceability because each disease result and spraying action is saved under the relevant plant record.

3.7.8. COMPLETE DISEASE AND SPRAYING DECISION WORKFLOW

The complete decision-making process shown below:

- Robot captures cucumber leaf image.
- AI model analyses image
- Class results generated
- Confidence score checked
- Is confidence acceptable?
  ├── No → Store for review, no spraying
  └── Yes → Continue
- Detected class checked.
- Healthy?
  ├── Yes → No action
  └── No → Continue.
- Water stress?
  ├── Yes → Send mobile alert, no spraying
  └── No → Continue.
- Downy mildew or powdery mildew?
  ├── Downy mildew → Select Mancozeb
  └── Powdery mildew → Select Hexaconazole
- Check severity level.

- Medium or high severity?
  ├── Yes → Activate targeted spraying
  └── No → Monitor and notify.
- Store results in Firebase.
- Display output in mobile application.

Overall, the disease detection and spraying decision logic allows the proposed system to move from simple disease recognition to practical field action. By combining AI classification, confidence filtering, severity estimation, disease-specific fungicide selection, Firebase-based decision transfer, and fail-safe spraying control, the system supports precision treatment in cucumber polytunnel cultivation. This helps reduce manual inspection, minimize unnecessary chemical usage, and improve timely crop health management.

3.8. LORA-BASED ENVIRONMENTAL MONITORING AND SENSOR ANOMALY DETECTION

The LoRa-based environmental monitoring module was developed to collect temperature and humidity readings from the cucumber polytunnel and transmit them wirelessly to the monitoring system. This module is important because environmental conditions directly affect cucumber growth, disease development, flowering, and fruit quality. In the proposed system, LoRa communication is used because it supports long-range and low-power transmission, which is suitable for polytunnel environments where continuous Wi-Fi coverage may not be available.

The monitoring unit consists of temperature and humidity sensors, a microcontroller, LoRa transmitter, LoRa receiver or gateway, Firebase cloud database, and mobile application. The sensor node collects temperature and humidity values from the polytunnel at selected time intervals. After collecting the data, the microcontroller prepares a data packet and sends it through the LoRa transmitter. The LoRa receiver receives the packet and uploads the values to Firebase. The mobile application then displays the latest environmental readings to the farmer.

The basic data flow of this module is:

- Temperature and humidity sensors
- Microcontroller
- LoRa transmitter
- LoRa receiver / gateway
- Firebase database
- Mobile application

Sensor anomaly detection was added to improve the reliability of the monitoring system. In smart agriculture, incorrect sensor readings can occur due to sensor faults, poor calibration, unstable power supply, communication errors, or environmental damage. If these incorrect readings are shown directly to the farmer, they may lead to wrong decisions. Therefore, the system compares readings from two temperature and humidity sensors before finalizing the monitoring status. The uploaded LoRa component explains that the system compares two sensor readings and identifies an anomaly when the difference exceeds a predefined threshold.

The anomaly calculation performed as follows:

$$\Delta T = |T1 - T2|$$
$$\Delta H = |H1 - H2|$$

Where T1 and T2 are the temperature readings from two sensors, and H1 and H2 are the humidity readings from two sensors. ΔT represents the temperature difference, while ΔH represents the humidity difference. If either difference is higher than the predefined threshold, the system marks the reading as abnormal and sends an anomaly alert to Firebase and the mobile application.

Condition	**System Status**	**Action**
Temperature difference within threshold	Normal	Display sensor values
Humidity difference within threshold	Normal	Display sensor values
Temperature difference exceeds threshold	Anomaly	Send temperature anomaly alert
Humidity difference exceeds threshold	Anomaly	Send humidity anomaly alert
Communication failure	Warning	Show sensor/gateway issue

Table 35 Sensor Anomaly Detection Logic

This method was selected because it is simple, explainable, and lightweight. It does not require high processing power or a large historical dataset. Therefore, it is suitable for a low-cost prototype system using microcontrollers, LoRa modules, Firebase, and a mobile application. However, the accuracy of this method depends on selecting suitable threshold values. If the threshold is too strict, false alerts may occur. If the threshold is too relaxed, actual sensor faults may not be detected.

Field	Description
Node ID	Identifies the sensor node
Temperature	Current temperature reading
Humidity	Current humidity reading
Temperature difference	Difference between two temperature readings
Humidity difference	Difference between two humidity readings
Anomaly status	Normal or anomaly
Timestamp	Time of data update

Table 36 LoRa Monitoring Data Fields

Overall, the LoRa-based monitoring and anomaly detection module improves the reliability of the smart cucumber polytunnel system. It allows farmers to monitor environmental conditions remotely while also checking whether the sensor readings are trustworthy. This supports better decision-making for crop health, irrigation, disease risk observation, and general polytunnel management.

3.9. CUCUMBER DETECTION AND RIPENESS ESTIMATION METHODOLOGY

The cucumber detection and ripeness estimation module was developed to identify cucumber fruits inside the polytunnel and classify them as **ripe** or **non-ripe** based on estimated physical size. This module supports harvest decision-making because cucumber fruits are often difficult to identify manually due to their green color, overlapping leaves, shadows, and complex plant backgrounds.

A **YOLOv8 object detection model** was used to detect cucumber fruits from captured images. The model was trained using **891 cucumber images**, and each cucumber fruit was annotated using bounding boxes. After training, the model detects cucumbers in new images and produces bounding box coordinates. These bounding boxes provide the pixel-based width and height of the cucumber, which are later used for size estimation.

The cucumber detection workflow follows these steps:

- Camera captures cucumber plant image

- YOLOv8 model detects cucumber fruit.
- Bounding box is generated.
- Pixel-based length and width are extracted.
- Values are passed to ripeness estimation logic.

Item	Description
Detection model	YOLOv8 object detection model
Dataset size	891 cucumber images
Annotation type	Bounding box annotation
Input	Cucumber plant image
Output	Cucumber bounding box and confidence score
Use of output	Ripeness estimation

Table 37 Cucumber Detection Model Details

Ripeness estimation was performed using **image-based dimensions and ultrasonic distance measurement**. Since salad cucumbers usually remain green even when they are ready for harvesting, color-based ripeness detection is not reliable. Therefore, this research uses physical size as the main harvest-readiness indicator. The uploaded component explains that cucumber ripeness is estimated by combining YOLOv8 bounding box dimensions with the distance measured by an ultrasonic sensor.

Pixel dimensions alone cannot give the actual cucumber size because the apparent size changes depending on the distance between the camera and the cucumber. Therefore, the ultrasonic sensor measures the distance from the robot or camera to the cucumber plant. Then, the system uses the bounding box size and distance value to estimate the real-world cucumber length and diameter.

The ripeness decision is made by comparing the estimated cucumber size with the selected ripe cucumber size range. In this system, a cucumber is considered ripe when the estimated length is around **15–16 cm** and the estimated diameter is around **2.5–3.0 cm**. If the estimated values are within this range, the cucumber is classified as ripe; otherwise, it is classified as non-ripe.

Measurement	Ripe Range
Estimated length	15–16 cm
Estimated diameter	2.5–3.0 cm
Final output	Ripe / non-ripe

Table 38 Ripeness Classification Logic

The result is stored in Firebase and displayed through the mobile application. This allows the farmer to view cucumber detection results and harvest-readiness status remotely. The same component confirms that Firebase and the React Native mobile app are used to display sensor readings, cucumber detection results, ripeness status, and anomaly alerts.

Field	Description
Cucumber detected	Yes / No
Confidence score	YOLOv8 detection confidence
Bounding box size	Pixel width and height
Distance value	Ultrasonic sensor reading
Estimated length	Approximate real-world cucumber length
Estimated diameter	Approximate real-world cucumber diameter
Ripeness status	Ripe / non-ripe

Table 39 Cucumber Ripeness Output Data

Overall, this module provides a practical method for harvest-readiness estimation in cucumber polytunnel cultivation. By combining YOLOv8 detection with ultrasonic distance measurement, the system can estimate cucumber size more effectively than using image pixels alone. This supports farmers by reducing manual inspection effort and providing a simple ripe/non-ripe decision through the mobile application.

3.10. NUTRITION MANAGEMENT METHODOLOGY

The nutrition management module was developed to support automated and stage-based fertigation for cucumber plants in the polytunnel. The main purpose of this module is to reduce dependence on fixed fertilizer schedules and manual nutrient preparation by combining real-time NPK monitoring, automated nutrient mixing, flower detection-based season switching, and mobile application supervision. The nutrition component consists of two main IoT subsystems: the **NPK Monitor** and the **NPK Mixer**, together with an image-based flower detection mechanism.

The **NPK Monitor** is responsible for measuring soil nutrient values near the cucumber root zone. It uses an NPK soil sensor, ESP32 microcontroller, mini-OLED display, battery pack, and data transmission interface. The NPK sensor measures nitrogen, phosphorus, and potassium values from the soil, while the ESP32 reads and processes these values. The OLED display shows the readings locally in the field, and the same values are made available through the mobile application for remote monitoring.

The **NPK Mixer** acts as the nutrient preparation and delivery subsystem. It contains three nutrient tubs, three solenoid pumps, three load cells, a mixing barrel, and a stirring mechanism. The three nutrient tubs store liquid nutrient sources for nitrogen, phosphorus, and potassium. When the system selects a nutrient profile, the relevant pumps are activated to dispense the required amount of each nutrient. Load cells are used to measure the dispensed quantities, and the nutrients are mixed inside the barrel before being delivered through the drip irrigation line.

The module uses **stage-based nutrient profiles** because cucumber nutrient requirements change according to the crop growth stage. In this system, three nutrient seasons are considered. The first two seasons follow predefined nutrient schedules, while the third season represents the flowering stage. When the current stage is identified, the system selects the relevant nutrient ratio and sends it to the NPK Mixer for preparation.

Flower detection is used as the main trigger for switching to the flowering-stage nutrient profile. When the YOLOv8 flower detection model identifies a cucumber flower, the system recognizes that the crop has entered the flowering stage. Then, the active nutrient season is updated, and the NPK Mixer changes the mixing percentages according to the flowering-stage profile. This makes the system more adaptive than a fully time-based fertilizer schedule.

The nutrition management workflow can be summarized as follows:

- NPK sensor reads soil nutrient values.
- ESP32 processes N, P, and K values
- Values are shown on OLED and mobile app.

- System checks active crop stage.
- Flower detection result is checked.
- If flower is detected, flowering season is activated.
- NPK ratio is selected.
- Mixer dispenses nutrients using pumps and load cells.
- Nutrients are mixed in the barrel.
- Final solution is delivered through drip irrigation.

Component	Function
NPK soil sensor	Measures nitrogen, phosphorus, and potassium values
ESP32	Reads sensor values and controls the module
OLED display	Shows local NPK readings
Nutrient tubs	Store N, P, and K nutrient solutions
Solenoid pumps	Dispense nutrient liquids
Load cells	Measure dispensed nutrient quantity
Mixing barrel	Holds and mixes the nutrient solution
Stirrer	Mixes nutrients evenly
Drip irrigation line	Deliver nutrient solutions to cucumber plants
Mobile application	Displays NPK values and nutrition status

Table 40 Nutrition Management Module Components

Condition	Active Nutrition Mode	System Action
No flower detected	Current scheduled season	Continuing current nutrient profile
Flower detected	Flowering season	Switch to flowering-stage NPK ratio
Manual override selected	Farmer-selected mode	Apply selected nutrient profile

Device/sensor issue	Warning mode	Notify farmer through mobile app

Table 41 Nutrition Season Switching Logic

The mobile application supports farmer supervision by displaying real-time NPK values, device status, active nutrient season, and mixer status. It also allows manual intervention when required. This is important because the system is designed as a farmer-supervised smart fertigation system rather than a fully hidden automatic process. The uploaded nutrition component confirms that the mobile app displayed NPK values, device status, season information, and manual override options during testing.

Overall, the nutrition management methodology connects nutrient sensing, crop-stage recognition, automated mixing, drip fertigation, and mobile monitoring into one workflow. This helps the proposed system provide more adaptive nutrient management for cucumber polytunnel cultivation.

3.11. FIREBASE AND MOBILE APPLICATION INTEGRATION

Firebase and the mobile application were integrated as the main communication and monitoring layer of the proposed smart cucumber polytunnel system. Since the system contains several modules, including robot, disease detection, LoRa monitoring, cucumber ripeness estimation, anomaly detection, and nutrition management, a central cloud platform was required to store and share data between these modules. Firebase was selected because it supports cloud-based data storage, real-time updates, and easy integration with mobile applications.

The Firebase database stores important system outputs such as robot status, RFID plant ID, scan events, disease detection results, spray decisions, environmental sensor readings, anomaly status, cucumber detection results, ripeness classification, NPK values, active nutrition season, and mixer status. In the robot module, Firebase is used for robot status updates, scanning logs, and decision transfer between the AI model and robot controller. In the LoRa and cucumber monitoring module, Firebase stores sensor readings, anomaly status, cucumber detection results, and ripeness status.

The React Native mobile application was developed as the farmer-facing interface of the system. The application displays real-time system outputs in a simple way so that farmers can monitor the polytunnel without directly checking the hardware or database. The mobile app shows temperature, humidity, anomaly alerts, cucumber detection status, ripe or non-ripe result, disease alerts, water stress alerts, NPK readings, nutrition season, and device status.

The mobile application also supports image capture in the robot workflow. A mobile phone camera is mounted on the vertical gantry, and the React Native app captures cucumber plant images when the robot creates a capture request. The captured images are processed by the AI model, and the final decision is returned through Firebase. This method keeps the ESP32 lightweight because image processing is handled outside the microcontroller.

For the nutrition management module, the mobile application works as both a monitoring and control interface. It displays real-time NPK values, device status, season information, and mixer status. It also supports manual override, allowing the farmer to manually adjust nutrient-related settings when required. This makes the system semi-automated, where automation supports the farmer but does not completely remove farmer supervision.

Data Category	**Stored in Firebase**	**Displayed in Mobile App**
Robot status	Movement, scan status, spray status	Robot current state
Plant identification	RFID value, plant ID	Current scanned plant
Disease detection	Class, confidence, severity	Disease/water stress alert
Spraying decision	Spray/no spray, fungicide type	Treatment status
Environmental data	Temperature and humidity	Tunnel condition
Anomaly detection	Normal/anomaly status	Sensor warning
Cucumber ripeness	Detected fruit, ripe/non-ripe result	Harvest readiness
Nutrition data	NPK values, active season, mixer status	Fertigation monitoring

Table 42 Firebase and Mobile Application Data Integration

The general data flow of the Firebase and mobile application integration is:

- Hardware / AI module
- Firebase database update

- Mobile application reads latest data.
- Farmer views status, alerts, and decisions
- Manual command can be sent if needed.

Overall, Firebase and the React Native mobile application connect the technical system with the farmer. Firebase acts as the central data-sharing platform, while the mobile application converts raw sensor values, AI outputs, and control decisions into readable information. This improves usability, supports remote monitoring, and makes the proposed smart cucumber polytunnel system more practical for real farming environments.

4. RESULTS AND DISCUSSION

4.1. INTRODUCTION

This chapter presents the **results and discussion** of the proposed integrated smart cucumber polytunnel management system. The system was evaluated to identify whether the developed prototype could perform the main functions described in the methodology chapter, including robot navigation, RFID-based plant identification, vertical scanning, AI-based disease and water stress detection, targeted spraying, LoRa-based environmental monitoring, cucumber ripeness estimation, sensor anomaly detection, NPK monitoring, automated nutrient mixing, flower-based season switching, Firebase integration, and mobile application visualization.

The main purpose of this evaluation was to check whether the individual modules worked correctly and whether they could operate together as one smart agriculture system. The RFID robot component was tested based on path following, RFID reading, plant position detection, vertical camera movement, image capture requests, spray decision handling, pump activation, and Firebase event logging. The individual robot results show that the robot workflow was evaluated from navigation to spraying and cloud logging, which supports the plant-level smart farming concept.

The AI-based disease and water stress detection module was evaluated using standard performance metrics such as accuracy, precision, recall, and F1-score. The testing focused on identifying healthy leaves, downy mildew, powdery mildew, and water stress conditions from cucumber leaf images. The disease detection results also supported real-time decision-making, where downy mildew and powdery mildew triggered spraying decisions, while water stress generated farmer alerts.

The LoRa and cucumber monitoring module was evaluated by testing temperature and humidity data transmission, YOLOv8-based cucumber detection, ultrasonic-assisted ripeness estimation, two-sensor anomaly detection, Firebase storage, and

mobile application display. The results confirmed whether the system could provide environmental monitoring, cucumber-level decision support, and sensor reliability checking through an integrated framework.

The nutrition management module was evaluated based on real-time NPK sensing, OLED display output, mobile application updates, automated nutrient mixing, drip-line delivery, and flower-triggered switching to the flowering nutrient profile. The results showed whether the system could move beyond fixed nutrient scheduling and support adaptive fertigation according to actual crop development.

Overall, this chapter discusses both the functional results and practical observations of the proposed system. The results are presented module by module and then discussed as a complete integrated smart cucumber polytunnel solution. This evaluation helps determine the feasibility, usefulness, limitations, and future improvement areas of the developed prototype for Sri Lankan protected cucumber cultivation.

4.2. RESULTS

The proposed integrated smart cucumber polytunnel management system was tested by evaluating each major module separately and then considering the overall system workflow. The evaluation focused on whether the system could collect plant and environmental data, process the data using AI and rule-based logic, perform required actions, and display the outputs through the mobile application.

The robot-based plant monitoring module successfully demonstrated the basic workflow of moving through the cucumber row, identifying plants using RFID, stopping at plant positions, moving the camera through the vertical scanning mechanism, capturing plant images, receiving a spray or no-spray decision, and storing the scan events in Firebase. This confirms that the robot can support plant-level monitoring instead of only general tunnel-level observation. The robot component was tested using modules such as ESP32 controller, MFRC522 RFID reader, IR sensors, stepper motor, mobile camera, Firebase, pump, and servo-controlled nozzle.

The disease and water stress detection module produced strong AI performance. The hybrid model, which combined YOLOv8 and Mask R-CNN, achieved better results than the baseline CNN model. The CNN model achieved an accuracy of **0.8920**, while the hybrid model achieved an accuracy of **0.9697**. The hybrid model also achieved **0.9190 precision**, **0.9024 recall**, and **0.9106 F1-score**, showing that it was more suitable for detecting cucumber disease and water stress under complex polytunnel conditions.

The cucumber detection and ripeness estimation module also showed positive results. The YOLOv8 cucumber detection model was trained using **891 cucumber images** and evaluated using object detection metrics. The model achieved **0.9190 precision**, **0.9024 recall**, **0.9697 mAP@0.5**, and **0.8704 mAP@0.5:0.95**. These results show that the model was able to detect cucumber fruits accurately and generate bounding boxes that could be used for size-based ripeness estimation.

The LoRa-based environmental monitoring module successfully transmitted temperature and humidity readings from the sensor node to the receiver/gateway. The anomaly detection method compared readings from two sensors and identified abnormal differences. This helped improve the reliability of environmental monitoring because faulty or inconsistent sensor readings could be detected before being shown to the farmer.

The nutrition management module successfully demonstrated real-time NPK sensing, OLED-based local display, mobile application update, season-based nutrient selection, flower-triggered switching to the flowering stage, automated nutrient mixing, drip irrigation delivery, post-fertigation monitoring, and manual override through the mobile application. These results confirm that the nutrition subsystem can support smart fertigation instead of fully manual nutrient preparation.

System Module	**Tested Function**	**Result**
Robot navigation	Line following and stopping near plant positions	Successful
RFID identification	Plant-level identification using RFID tags	Successful
Vertical scanning	Camera movement along plant height	Successful
Disease and water stress detection	Healthy, downy mildew, powdery mildew, water stress classification	Successful
Targeted spraying	Spray/no-spray decision and pump activation	Successful
Cucumber detection	YOLOv8 fruit detection	Successful
Ripeness estimation	Size-based ripe/non-ripe classification	Successful
LoRa monitoring	Temperature and humidity data transmission	Successful

Sensor anomaly detection	Detection of abnormal sensor differences	Successful
NPK monitoring	Real-time NPK values reading and display	Successful
NPK mixing	Automated nutrient preparation	Successful
Flower-based season switching	Switching to flowering nutrient profile	Successful
Firebase integration	Cloud storage of system outputs	Successful
Mobile application	Display of alerts, values, and system status	Successful

Table 43 Summary of Main System Results

Overall, the results show that the proposed system was functionally feasible as a smart cucumber polytunnel management prototype. Each module achieved its main function, and the integrated workflow demonstrated that robotics, IoT sensing, AI detection, LoRa communication, automated spraying, nutrition control, Firebase, and mobile monitoring can be combined into one practical smart agriculture system.

4.3. RESEARCH FINDINGS

Based on the obtained results, several important research findings can be identified from the proposed integrated smart cucumber polytunnel management system. These findings show that the developed system can combine robotics, IoT, LoRa communication, computer vision, cloud storage, mobile application support, targeted spraying, ripeness estimation, and nutrition management into one practical smart agriculture framework.

The first finding is that **RFID-based plant identification can successfully support plant-level tracking** in cucumber polytunnel farming. By assigning an RFID tag to each plant, the system can identify the exact plant being scanned and store scan results, disease outputs, spray decisions, and plant history under the correct plant record. This improves traceability compared with general tunnel-level monitoring systems. The robot component also identifies RFID plant identification, vertical scanning, Firebase logging, and targeted spraying as key improvements over traditional manual inspection and blanket spraying.

The second finding is that **vertical scanning improves image coverage for cucumber plants**. Since cucumber plants grow upward and leaves appear at different heights, a fixed camera cannot capture the full plant properly. The vertical gantry allowed the camera to capture plant images from multiple levels, improving the possibility of detecting leaf disease, water stress, cucumber fruits, and flowers.

However, the results also showed that vertical scanning increases scan time, which should be improved in future versions.

The third finding is that **AI-based image analysis is effective for cucumber disease and water stress detection**. The hybrid model achieved better results than the baseline CNN model, with higher accuracy, precision, recall, and F1-score. The system was able to classify cucumber leaf conditions into healthy, downy mildew, powdery mildew, and water stress classes. This confirms that image-based detection can reduce dependence on manual observation and support early crop health monitoring in polytunnel conditions.

The fourth finding is that **AI detection can relate to targeted spraying decisions**. The system does not stop disease detection; it converts the result into practical action. Downy mildew detection leads to Mancozeb spraying, powdery mildew detection leads to Hexaconazole spraying, water stress detection generates a mobile alert, and healthy detection performs no action. This supports efficient resource usage and reduces unnecessary chemical application.

The fifth finding is that **YOLOv8 can effectively detect cucumber fruits under polytunnel image conditions**. The cucumber detection model achieved strong performance, with precision of 0.9190, recall of 0.9024, mAP@0.5 of 0.9697, and mAP@0.5:0.95 of 0.8704. This confirms that YOLOv8 is suitable for detecting cucumber fruits even when fruits are visually similar to leaves and stems.

The sixth finding is that **size-based ripeness estimation is more suitable for salad cucumber than color-based ripeness detection**. Since salad cucumbers usually remain green when ready for harvesting, color alone is not reliable. The system used YOLOv8 bounding box dimensions together with ultrasonic distance measurements to estimate cucumber size and classify harvest readiness. This shows that cucumber detection can support decision-making beyond simple object localization.

The seventh finding is that **LoRa communication is suitable for environmental monitoring in cucumber polytunnels**. The system successfully transmitted temperature and humidity values from the sensor node to the receiver and updated them in Firebase. Since temperature and humidity readings are small data packets, LoRa is appropriate for low-power and long-range monitoring where Wi-Fi coverage may be limited.

The eighth finding is that **two-sensor comparison can improve sensor reliability**. By comparing two temperature and humidity readings, the system can identify abnormal differences and detect sensor faults or inconsistent readings. This is important because smart farming decisions depend heavily on accurate sensor data. The selected threshold-based method is simple, lightweight, and suitable for a low-cost prototype.

The ninth finding is that **real-time NPK monitoring and automated nutrient mixing can be integrated into a cucumber polytunnel fertigation workflow**. The NPK Monitor successfully measured soil macronutrient values, while the NPK Mixer prepared and delivered nutrient solutions using pumps, load cells, and a mixing barrel. This shows that nutrient management can be supported by live nutrient observation and automated preparation instead of only fixed schedules.

The tenth finding is that **flower detection can be used as a biological trigger for nutrition season switching**. When cucumber flowers were detected, the system switched to the flowering-stage nutrient profile. This makes the nutrition process more adaptive because it responds to actual plant development rather than only depending on crop age or predefined schedules.

The final finding is that **Firebase and React Native mobile applications improve farmer usability**. Firebase allowed the system to store and update sensor readings, robot status, disease results, spray decisions, cucumber ripeness outputs, anomaly alerts, and nutrition data. The mobile application presented these outputs in a user-friendly way, allowing the farmer to monitor the system remotely and manually intervene when required.

Area	**Main Finding**
RFID plant identification	Supports plant-level tracking and scan history
Vertical scanning	Improves image coverage of tall cucumber plants
Disease and water stress detection	Hybrid AI model improves cucumber leaf health classification
Targeted spraying	Converts disease detection into practical spray/no-spray action
Cucumber detection	YOLOv8 detects cucumber fruits effectively
Ripeness estimation	Size-based estimation is practical for salad cucumber harvest readiness
LoRa monitoring	Suitable for low-power environmental data transmission
Sensor anomaly detection	Two-sensor comparison improves reliability of readings
NPK monitoring	Enables real-time nutrient observation

NPK mixing	Supports automated stage-based nutrient preparation
Flower detection	Triggers flowering-stage nutrition profile
Firebase and mobile app	Improves remote monitoring and farmer supervision

Table 44 Summary of Research Findings

Overall, the research findings confirm that the proposed system is functionally feasible as an integrated smart cucumber polytunnel management prototype. The system successfully connects plant-level identification, robotic scanning, AI-based crop analysis, environmental monitoring, anomaly detection, nutrition control, cloud storage, and mobile visualization into one coordinated workflow.

4.4. DISCUSSION

The results show that the proposed system is a feasible prototype for smart cucumber polytunnel management. The main strength of the system is that it does not focus on only one task, such as disease detection or environmental monitoring. Instead, it combines robotic movement, RFID-based plant identification, vertical scanning, AI-based crop analysis, LoRa communication, sensor anomaly detection, nutrition management, Firebase storage, and mobile application monitoring into one integrated workflow. This supports the original research aim of developing a practical smart agriculture solution for cucumber cultivation under Sri Lankan polytunnel conditions.

The robot module showed that plant-level monitoring is possible when RFID identification is combined with vertical scanning and Firebase logging. This is important because many existing smart farming systems monitor only general tunnel conditions, but they do not identify individual plants or record plant-level treatment history. In this system, each plant can be linked with its own scan result, disease result, and spray action. The uploaded robot component also highlights that the robot workflow includes line following, RFID detection, vertical scanning, AI decision receiving, spray actuation, and Firebase event storage.

The disease and water stress detection module also produced useful results. The hybrid model performed better than the baseline CNN model and was able to classify healthy leaves, downy mildew, powdery mildew, and water stress conditions. This is valuable because manual disease identification can be slow and depends heavily on farmer experience. The disease component explains that cucumber diseases and water stress can reduce plant growth, fruit quality, and yield, and that image-based analysis can support earlier identification.

Another key point is the connection between AI detection and real action. The system does not only detect disease; it also converts the result into spraying or alert decisions. Downy mildew and powdery mildew trigger fungicide spraying, while water stress generates a farmer alert. This makes the system more useful than a simple image classification model because it supports practical field-level decision-making.

The LoRa monitoring and anomaly detection results show that environmental data can be collected and transferred wirelessly in a low-cost way. The two-sensor comparison method is also useful because it checks whether sensor readings are reliable before they are shown to the farmer. However, the accuracy of this anomaly method depends on suitable threshold values. If the threshold is too strict, the system may generate false alerts. If the threshold is too loose, real abnormal readings may not be detected.

The cucumber detection and ripeness estimation module supports harvest-related decision-making. Since cucumber salad does not show a strong color change when it becomes ready for harvesting, size-based ripeness estimation is more practical. The use of YOLOv8 bounding boxes together with ultrasonic distance measurement provides a simple method to estimate cucumber size and classify ripe or non-ripe status.

The nutrition management module also showed practical value. The NPK Monitor provided real-time nutrient readings, while the NPK Mixer converted the selected nutrient profile into an actual fertigation process. Flower detection-based season switching is especially important because it allows the system to change the nutrient profile according to actual plant development rather than depending only on fixed schedules. The nutrition component confirms that the module successfully demonstrated NPK sensing, automated nutrient mixing, controlled delivery, flower-triggered nutrient switching, and mobile monitoring, while also noting the need for better calibration and field validation.

The Firebase and mobile application integration improved the usability of the complete system. The farmer can view robot status, disease alerts, environmental readings, anomaly status, cucumber ripeness status, NPK values, and nutrition status through the mobile application. This is important because a smart agriculture system should not only collect data but also present it in a way that farmers can understand and use.

However, several limitations were identified. The vertical scanning process increases the time required to scan each plant, which may reduce scalability for larger farms. The system also depends on external computing for AI inference, which can introduce delay and reduce full autonomy. In addition, model performance may be

affected by lighting variation, overlapping leaves, camera angles, and background complexity. The nutrition module also requires further calibration because NPK readings, pump flow rate, load-cell accuracy, and mixing uniformity can affect final fertigation accuracy.

Overall, the discussion shows that the proposed system successfully demonstrates the core functions of an integrated smart cucumber polytunnel platform. It reduces dependence on manual observation, supports early detection, improves plant-level traceability, enables targeted spraying, assists harvest-readiness estimation, supports adaptive nutrient management, and improves farmer monitoring through mobile access. Although further improvements are needed before large-scale deployment, the prototype provides a strong foundation for future smart agriculture systems in Sri Lankan protected cultivation.

4.5. SUMMARY

This chapter presented the results, research findings, and discussion of the proposed integrated smart cucumber polytunnel management system. The system was evaluated as a combined prototype consisting of robot-based plant monitoring, RFID plant identification, vertical image scanning, AI-based disease and water stress detection, targeted spraying, LoRa-based environmental monitoring, cucumber ripeness estimation, sensor anomaly detection, NPK monitoring, automated nutrient mixing, flower-based season switching, Firebase integration, and mobile application visualization.

The results showed that the robot module successfully supported plant-level monitoring by combining line-following navigation, RFID-based plant identification, vertical scanning, Firebase communication, and spray decision handling. This confirmed that the system can move beyond general tunnel-level monitoring and store scan and treatment records for individual cucumber plants. The robot workflow supported the main idea of a plant-level sense–decide–act system.

The AI-based disease and water stress detection module showed strong performance in identifying healthy leaves, downy mildew, powdery mildew, and water stress conditions. The hybrid model performed better than the baseline approach and provided useful outputs for both classification and decision-making. These results confirm that computer vision can support early crop health monitoring and reduce dependence on manual inspection.

The system also demonstrated that AI results can be connected to real actions. Downy mildew and powdery mildew detections were linked with targeted fungicide spraying, while water stress was handled through farmer alerts. This supports

precision treatment because the robot sprays only when disease is detected, instead of applying chemicals to all plants.

The LoRa-based monitoring module successfully supported wireless transmission of temperature and humidity readings. The anomaly detection method improved reliability by comparing values from two sensors and identifying abnormal differences. This helped ensure that environmental readings shown to the farmers were more trustworthy.

The cucumber detection and ripeness estimation module showed that YOLOv8-based fruit detection, combined with ultrasonic distance measurement, can support size-based harvest-readiness estimation. This is suitable for salad cucumber because color-based ripeness detection is less reliable when fruits remain green at harvest stage.

The nutrition management module successfully combined NPK monitoring, automated nutrient mixing, flower detection-based season switching, and mobile application supervision. This confirmed that nutrient management can be made more adaptive by responding to real plant development rather than depending only on fixed fertilizer schedules.

Overall, the findings show that the proposed system is functionally feasible as a smart cucumber polytunnel management prototype. The system supports improved plant monitoring, reduced manual effort, early disease identification, targeted spraying, environmental observation, sensor reliability checking, harvest-readiness support, automated nutrition control, and farmer-friendly mobile supervision.

However, the system still has some limitations. The prototype requires further large-scale testing, improved mechanical durability, better sensor calibration, faster AI inference, stronger field validation, and more training data under different real polytunnel conditions. These limitations can be addressed in future development before commercial deployment.

In conclusion, this chapter confirms that the proposed integrated system provides a strong foundation for smart cucumber polytunnel farming in Sri Lanka. The next chapter presents the conclusion, main contributions, limitations, and future enhancements of the research.

4.6. CONCLUSION

This research presented an integrated smart cucumber polytunnel management system designed to improve crop monitoring, disease management, environmental observation, harvest-readiness estimation, and nutrition control. The proposed system combined robotics, IoT, LoRa communication, RFID-based plant identification,

computer vision, Firebase cloud storage, and mobile application supervision into one coordinated smart agriculture prototype.

The developed robot module successfully supported plant-level monitoring by using line-following navigation, RFID-based plant identification, vertical camera scanning, Firebase communication, and targeted spraying. This allowed each cucumber plant to be identified separately, scanned from different heights, and linked with its own scan records and treatment history. This approach improves traceability compared with traditional manual inspection and general tunnel-level monitoring systems.

The AI-based disease and water stress detection module showed that cucumber leaf conditions such as healthy leaves, downy mildew, powdery mildew, and water stress can be detected using image-based analysis. The hybrid model supported better detection performance and helped convert plant health results into practical actions. Disease detections were connected to targeted fungicide spraying, while water stress was handled through farmer alerts.

The LoRa-based monitoring and cucumber ripeness estimation module demonstrated that temperature and humidity values can be transmitted wirelessly and displayed through the mobile application. The sensor anomaly detection method improved reliability by identifying abnormal differences between sensor readings. In addition, YOLOv8-based cucumber detection and ultrasonic-assisted size estimation supported ripe and non-ripe classification for harvest decision-making.

The nutrition management module successfully demonstrated real-time NPK monitoring, automated nutrient mixing, flower detection-based season switching, and mobile application supervision. This allowed the system to move away from fully fixed nutrient schedules and support crop-stage-based fertigation decisions.

Overall, the proposed system achieved its main aim by demonstrating a functional smart cucumber polytunnel management prototype. The system can reduce manual inspection effort, support early disease detection, reduce unnecessary chemical spraying, improve environmental monitoring, assist harvest-readiness decisions, and support adaptive nutrient management. Although further improvements are required before large-scale deployment, the research provides a strong foundation for developing low-cost smart agriculture solutions for cucumber polytunnel farming in Sri Lanka.

4.7. MAIN RESEARCH CONTRIBUTIONS

The main contribution of this research is the development of an integrated smart cucumber polytunnel management system that combines multiple smart agriculture technologies into one coordinated prototype. Instead of focusing on only disease

detection, irrigation, or environmental monitoring, this system connects plant-level identification, robotic scanning, AI-based analysis, wireless sensing, automated spraying, nutrition management, cloud storage, and mobile supervision.

The first contribution is the **RFID-based plant-level monitoring approach**. Each cucumber plant can be identified separately using an RFID tag, allowing the system to store scan records, disease results, spraying actions, and plant history under the correct plant ID. This improves traceability and supports precision crop management compared with general tunnel-level monitoring.

The second contribution is the **robot-based vertical scanning mechanism**. Since cucumber plants grow vertically, the robot-mounted camera can move along the plant height and capture images from different levels. This improves plant coverage and supports more accurate image-based monitoring.

The third contribution is the **AI-based cucumber disease and water stress detection module**. The system can identify healthy leaves, downy mildew, powdery mildew, and water stress conditions. These outputs are used not only for monitoring but also for practical decision-making.

The fourth contribution is the **targeted fungicide spraying mechanism**. Instead of blanket spraying all plants, the system sprays only when disease is detected. Downy mildew and powdery mildew detections trigger fungicide spraying, while water stress generates a farmer alert without spraying.

The fifth contribution is the **LoRa-based environmental monitoring and anomaly detection module**. Temperature and humidity values are transmitted wirelessly, while abnormal sensor differences are detected using a two-sensor comparison method. This improves the reliability of environmental monitoring.

The sixth contribution is the **YOLOv8-based cucumber detection and ripeness estimation method**. The system detects cucumber fruits and uses ultrasonic distance measurement with bounding box dimensions to estimate harvest readiness.

The seventh contribution is the **automated nutrition management module**. The system measures NPK values, prepares nutrient solutions using an automated mixer, and changes nutrient profiles based on flower detection. This supports adaptive fertigation instead of depending only on fixed schedules.

The final contribution is the **Firebase and React Native mobile application integration**. All-important outputs, including disease alerts, sensor readings, ripeness status, NPK values, robot status, and spraying decisions, are displayed to the farmer through a mobile interface.

Contribution Area	Research Contribution
Plant identification	RFID-based plant-level tracking
Robot system	Autonomous movement and vertical plant scanning
Disease detection	AI-based detection of cucumber diseases and water stress
Spraying	Targeted fungicide spraying based on AI decision
Environmental monitoring	LoRa-based temperature and humidity monitoring
Sensor reliability	Two-sensor anomaly detection method
Ripeness estimation	YOLOv8 and ultrasonic-based harvest-readiness estimation
Nutrition management	NPK monitoring, automated mixing, and flower-based season switching
Mobile support	Firebase and React Native farmer monitoring interface

Table 45 Summary of Main Contributions

4.8. LIMITATIONS

Although the proposed smart cucumber polytunnel management system successfully demonstrated the main research objectives, several limitations were identified during development and testing. These limitations should be considered before applying the system in large-scale commercial farming environments.

The first limitation is that the system was developed as a **prototype-level solution**. The robot, spraying unit, LoRa monitoring, AI detection, and nutrition management modules were tested to verify functional feasibility. Therefore, long-term durability, continuous field operation, and large-scale commercial reliability still require further testing.

The second limitation is related to the **AI datasets**. Although disease detection, cucumber detection, and flower detection models produced useful results, the datasets were limited compared with real-world farming variation. More images are needed under different lighting conditions, plant ages, camera angles, disease stages, and background conditions to improve model generalization.

The third limitation is the **vertical scanning time**. Since the robot captures images from multiple plant heights, scanning each plant takes more time than a fixed camera

system. This may reduce efficiency when the system is used in a large polytunnel with many plants. Future improvements should focus on faster scanning and optimized camera movement.

Another limitation is the **dependency on external AI processing**. In the current prototype, heavy image processing is handled outside the ESP32 controller. This is practical for testing, but it may create delays in real-time operation. Edge computing devices such as Raspberry Pi, Jetson Nano, or similar AI hardware could improve autonomy.

The LoRa-based monitoring module also has limitations. Although LoRa is suitable for low-power wireless transmission, signal strength can be affected by distance, obstacles, antenna placement, and environmental conditions. Large-scale deployment would require proper gateway placement and communication testing.

The nutrition management module requires further **sensor and actuator calibration**. NPK sensor readings, pump flow rate, load-cell accuracy, and mixing uniformity can affect the final nutrient solution. Therefore, the NPK monitoring and mixing unit must be calibrated carefully before using it for real crop production.

The spraying system also needs improvement before field deployment. The current targeted spraying mechanism works as a prototype, but nozzle direction, spray coverage, fungicide dosage, pump pressure, and chemical safety should be tested more deeply in real polytunnel conditions.

Limitation Area	**Description**
Prototype level	System requires more long-term field testing
Dataset size	More real polytunnel images are needed
Scanning time	Vertical scanning increases time per plant
AI processing	External processing may cause delay
LoRa communication	Signals can vary with distance and obstacles
NPK calibration	Nutrient readings and mixing accuracy need improvement
Spraying accuracy	Nozzle direction and dosage need further testing
Mechanical durability	Robots need stronger protection against humidity and water
Large-scale use	System needs testing in bigger polytunnels

Table 46 Summary of System Limitations

Overall, these limitations do not reduce the value of the prototype, but they show the areas that must be improved before commercial deployment. The developed system proves the concept, while future work should focus on reliability, accuracy, durability, and scalability.

4.9. FUTURE ENHANCEMENTS

Although the proposed system successfully demonstrated the main functions of a smart cucumber polytunnel management prototype, several improvements can be made in future versions to increase accuracy, reliability, scalability, and commercial suitability.

The first future enhancement is to improve the **AI model performance** using larger and more diverse datasets. More cucumber leaf, fruit, and flower images should be collected under different lighting conditions, disease stages, camera angles, and real polytunnel backgrounds. This will help the disease detection, water stress detection, cucumber detection, and flower detection models perform better in practical field environments.

The second enhancement is to use **edge AI processing**. In the current prototype, heavy AI processing is handled externally. In the future, devices such as Raspberry Pi, NVIDIA Jetson Nano, or similar edge computing boards can be added to the robot. This will reduce decision delays and make the robot more independent.

The third improvement is to increase the **speed and efficiency of vertical scanning**. The scanning mechanism can be optimized using faster motors, better camera positioning, and fewer but more effective scan points. This will help the robot inspect more plants in less time.

The fourth enhancement is to improve the **mechanical design and durability** of the robot. Since polytunnels can be humid and wet, the robot body, electronics, wiring, and sensors should be protected using waterproof and dust-resistant enclosures. This will improve long-term field operation.

The fifth future improvement is to develop a more accurate **spraying control system**. The nozzle direction, spray pressure, spray duration, and fungicide dosage should be calibrated more precisely. In future versions, the system can also support disease-severity-based spraying, where the amount of fungicide depends on the infection level.

The sixth enhancement is to improve the **nutrition management system**. NPK sensors, load cells, pumps, and mixing ratios should be calibrated with more accuracy. The system can also be improved by adding pH and EC sensors to monitor the nutrient solution quality before fertigation.

The seventh improvement is to expand the **mobile application**. Future versions can include historical graphs, plant-level disease history, harvest prediction, notification settings, manual robot control, multi-tunnel monitoring, and farmer reports. This will make the system more useful for real farm management.

The final enhancement is to conduct **large-scale field testing**. The system should be tested in multiple cucumber polytunnels over several crop cycles. This will help evaluate long-term performance, farmer usability, yield improvement, chemical reduction, and commercial feasibility.

Area	Future Improvement
AI models	Train with larger and more diverse datasets
Processing	Add edge AI device for faster decisions
Robot scanning	Improve scanning speed and camera positioning
Mechanical design	Add waterproof and dust-resistant protection
Spraying	Improve nozzle control, dosage, and spray accuracy
Nutrition	Add better calibration, pH, and EC monitoring
Mobile app	Add history, graphs, reports, and multi-tunnel support
Field testing	Test in larger polytunnels over multiple crop cycles

Table 47 Future Enhancements

Overall, these future enhancements can transform the current prototype into a more reliable and scalable smart farming system suitable for real cucumber polytunnel cultivation.

4.10. FINAL SUMMARY

This chapter concluded the research by summarizing the overall outcome of the proposed integrated smart cucumber polytunnel management system. The developed prototype successfully combined robotics, IoT, LoRa communication, RFID-based plant identification, AI-based crop analysis, targeted spraying, cucumber ripeness estimation, nutrition management, Firebase cloud storage, and mobile application supervision into one smart agriculture framework.

The system showed that cucumber polytunnel farming can be improved by moving from manual observation and fixed management practices toward automated, data-driven, and plant-level decision-making. RFID identification and vertical scanning supported individual plant monitoring, while AI-based disease and water stress detection helped identify plant health problems at an early stage. The targeted spraying mechanism reduced the need for blanket spraying by activating treatment only when disease was detected.

The LoRa-based monitoring module supported remote environmental observation, while sensor anomaly detection improved the reliability of temperature and humidity readings. The cucumber detection and ripeness estimation module provided harvest-readiness support using YOLOv8 detection and ultrasonic distance measurement. The nutrition management module improved fertigation by combining NPK monitoring, automated nutrient mixing, and flower-based season switching.

The research also identified several limitations, including prototype-level testing, limited datasets, scanning time, dependency on external AI processing, calibration requirements, and the need for stronger field validation. However, these limitations can be addressed through future improvements such as larger datasets, edge AI processing, improved mechanical design, better spraying accuracy, enhanced nutrition calibration, and large-scale field trials.

In conclusion, the proposed system provides a strong foundation for developing a practical smart cucumber polytunnel management solution for Sri Lankan protected agriculture. It demonstrates how multiple smart farming technologies can work together to reduce manual workload, improve crop monitoring, support timely decision-making, reduce unnecessary chemical usage, and improve overall cultivation management.

[1] S. N. A. Jayathilaka, K. K. L. B. Adikaram, and H. K. M. S. Kumarasinghe, “Effectiveness of IoT based automation system for salad cucumber (*Cucumis sativus*) cultivation in protected house under Sri Lankan conditions,” *Tropical Agricultural Research and Extension*, vol. 23, no. 3–4, pp. 80–91, 2020, doi: 10.4038/tare.v23i3-4.5500.

[2] M. S. Jayathilaka, K. K. L. B. Adikaram, H. K. M. S. Kumarasinghe, and G. Y. Jayasinghe, “Investigation of the potential to introduce Internet of Things based micro-climate controlling system for salad cucumber (*Cucumis sativus*) in conventional protected houses,” *Sri Lankan Journal of Agriculture and Ecosystems*, vol. 4, no. 2, pp. 25–40, 2022, doi: 10.4038/sljae.v4i2.83.

[3] N. Jaliyagoda et al., “Internet of Things (IoT) for smart agriculture: Assembling and assessment of a low-cost IoT system for polytunnels,” *PLOS ONE*, vol. 18, no. 5, 2023, doi: 10.1371/journal.pone.0278440.

[4] S. G. Vougioukas, “Agricultural robotics,” *Annual Review of Control, Robotics, and Autonomous Systems*, 2019, doi: 10.1146/annurev-control-053018-023617.

[5] X. Shi et al., “State-of-the-art Internet of Things in protected agriculture,” *Sensors*, vol. 19, no. 8, 2019, doi: 10.3390/s19081833.

[6] L. Ruiz-Garcia and L. Lunadei, “The role of RFID in agriculture: Applications, limitations and challenges,” *Computers and Electronics in Agriculture*, vol. 79, no. 1, 2011, doi: 10.1016/j.compag.2011.08.010.

[7] Y. Y. Han et al., “A crop trait information acquisition system with multitag-based identification technologies for breeding precision management,” *Computers and Electronics in Agriculture*, vol. 135, 2017, doi: 10.1016/j.compag.2017.01.004.

[8] N. Jayasuriya, Y. Guo, W. Hu, and O. Ghannoum, “Machine vision based plant height estimation for protected crop facilities,” *Computers and Electronics in Agriculture*, vol. 218, 2024, doi: 10.1016/j.compag.2024.108669.

[9] Ultralytics, “Explore Ultralytics YOLOv8,” *Ultralytics YOLO Documentation*, Accessed: Apr. 26, 2026.

[10] X. Wang and J. Liu, “Vegetable disease detection using an improved YOLOv8 algorithm in the greenhouse plant environment,” *Scientific Reports*, vol. 14, no. 1, 2024, doi: 10.1038/s41598-024-54540-9.

[11] J. Xie, X. Xie, W. Xie, and Q. Xie, “An improved YOLOv8-based method for detecting pests and diseases on cucumber leaves in natural backgrounds,” *Sensors*, vol. 25, no. 5, 2025, doi: 10.3390/s25051551.

[12] J. Ma, K. Du, F. Zheng, L. Zhang, Z. Gong, and Z. Sun, “A recognition method for cucumber diseases using leaf symptom images based on deep convolutional neural network,” *Computers and Electronics in Agriculture*, vol. 154, pp. 18–24, 2018, doi: 10.1016/j.compag.2018.08.048.

[13] S. Li, K. Li, Y. Qiao, and L. Zhang, “A multi-scale cucumber disease detection method in natural scenes based on YOLOv5,” *Computers and Electronics in Agriculture*, vol. 202, 2022, doi: 10.1016/j.compag.2022.107363.

[14] A. Upadhyay, N. S. Chandel, K. P. Singh, et al., “Deep learning and computer vision in plant disease detection: A comprehensive review of techniques, models, and trends in precision agriculture,” *Artificial Intelligence Review*, vol. 58, 2025, doi: 10.1007/s10462-024-11100-x.

[15] N. Paul, G. C. Sunil, D. P. Horvath, and X. Sun, “Deep learning for plant stress detection: A comprehensive review of technologies, challenges, and future

directions," *Computers and Electronics in Agriculture*, vol. 229, 2025, doi: 10.1016/j.compag.2024.109734.

[16] S. B. Cho et al., "Recent methods for evaluating crop water stress using AI techniques: A review," *Sensors*, vol. 24, no. 19, 2024, doi: 10.3390/s24196313.

[17] H. Li et al., "Design of field real-time target spraying system based on improved YOLOv5," *Frontiers in Plant Science*, vol. 13, 2022, doi: 10.3389/fpls.2022.1072631.

[18] F. Yang, K. Wang, Y. Han, and Z. Qiao, "A cloud-based digital farm management system for vegetable production process management and quality traceability," *Sustainability*, vol. 10, no. 11, 2018, doi: 10.3390/su10114007.

[19] N. Katal, M. Rzanny, P. Mäder, and J. Wäldchen, "Deep learning in plant phenological research: A systematic literature review," *Frontiers in Plant Science*, vol. 13, 2022, doi: 10.3389/fpls.2022.805738.

[20] X. Xu et al., "Cucumber flower detection based on YOLOv5s-SE7 within greenhouse environments," *IEEE Access*, vol. 11, pp. 64358–64369, 2023, doi: 10.1109/ACCESS.2023.3286545.

[21] R. Singh, A. Khan, L. Seneviratne, et al., "Deep learning approach for detecting tomato flowers and buds in greenhouses on 3P2R gantry robot," *Scientific Reports*, vol. 14, 2024, doi: 10.1038/s41598-024-71013-1.

[22] U. Ahmad, A. Alvino, and S. Marino, "Solar fertigation: A sustainable and smart IoT-based irrigation and fertilization system for efficient water and nutrient management," *Agronomy*, vol. 12, no. 5, 2022, doi: 10.3390/agronomy12051012.

[23] A. R. Zubair and T. Adebiyi, "Development of an IoT-based automatic fertigation system," *Journal of Agriculture Science and Technology*, vol. 21, no. 3, pp. 4–21, 2022, doi: 10.4314/jagst.v21i3.2.

[24] F. Idris et al., "IoT-based fertigation system for agriculture," *Bulletin of Electrical Engineering and Informatics*, vol. 13, no. 3, pp. 1574–1581, 2024, doi: 10.11591/eei.v13i3.6829.

[25] G. Sun et al., "Design and testing of a nutrient mixing machine for greenhouse fertigation," *Engineering in Agriculture, Environment and Food*, vol. 8, no. 2, pp. 114–121, 2015, doi: 10.1016/j.eaef.2014.12.001.

[26] Google Firebase, "Cloud Firestore documentation," *Firebase Documentation*, Accessed: Apr. 26, 2026.

[27] Department of Agriculture, Sri Lanka, "HORDI Crop – Cucumber," Accessed: Apr. 26, 2026.

[28] D. Shivaraj, P. Prasanth, D. Lakshminarayana, and T. Ramesh, “Studies on the effect of training systems on cucumber (*Cucumis sativus* L.) cv. Malini grown under protected conditions,” *Current Journal of Applied Science and Technology*, 2020, doi: 10.9734/cjast/2020/v39i4831276.